



Optimized Deep Learning - Jean Zay

Training and large batches



INSTITUT DU
DÉVELOPPEMENT ET DES
RESSOURCES EN
INFORMATIQUE
SCIENTIFIQUE



Loss Landscape

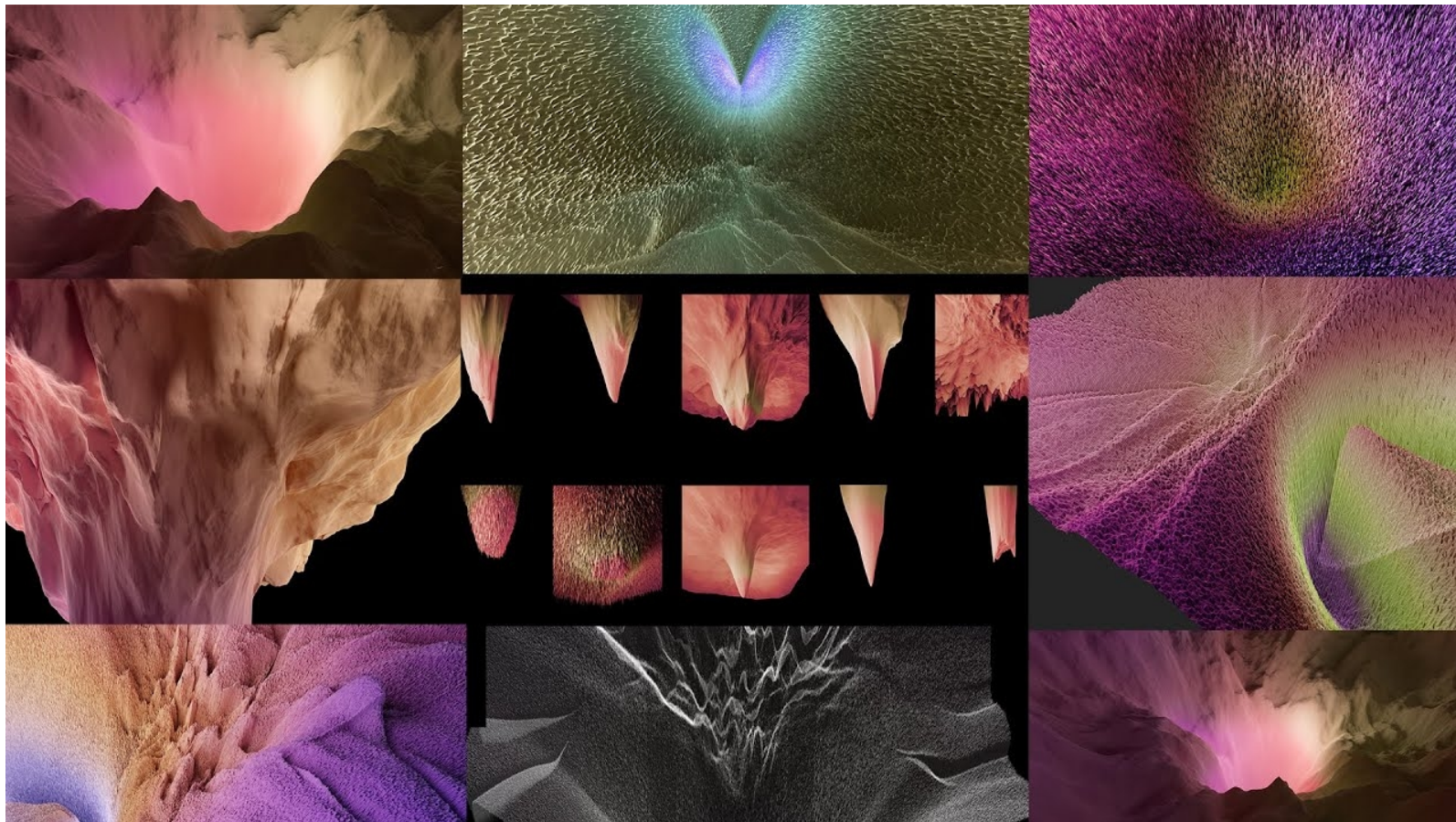
Loss Landscape ◀

Residual Learning ◀

Initialization ◀

Loss Landscape

<https://losslandscape.com/>

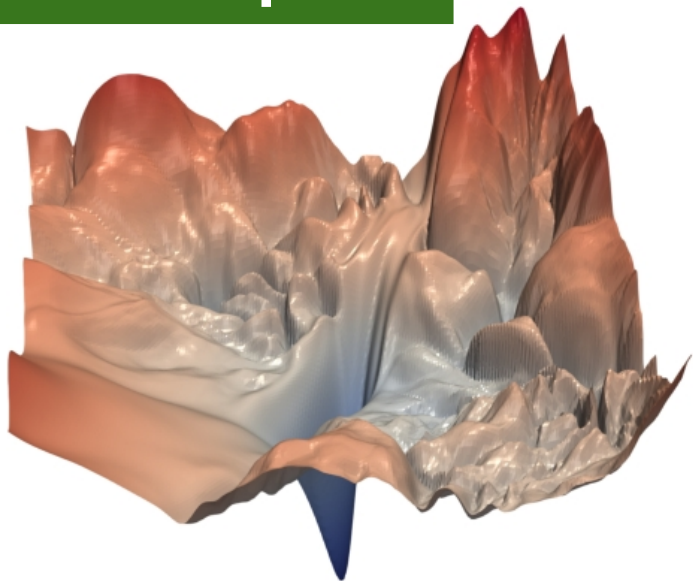


Loss Landscape

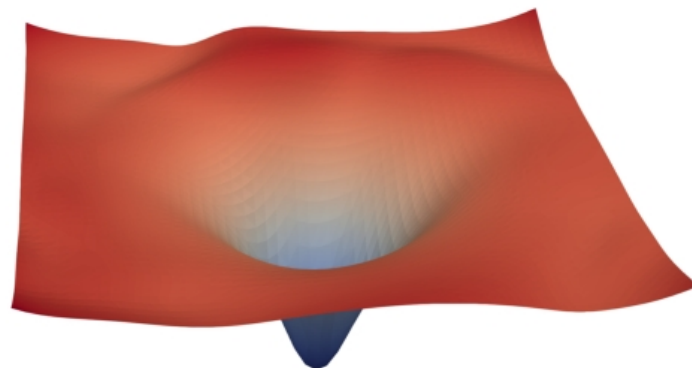
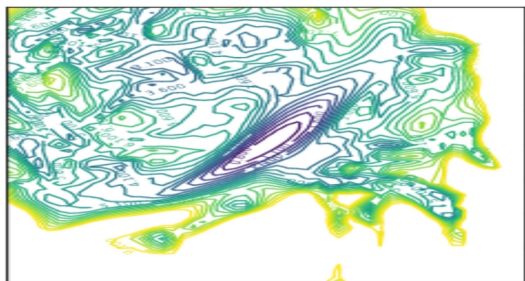
<https://arxiv.org/pdf/1712.09913.pdf>

Residual Learning

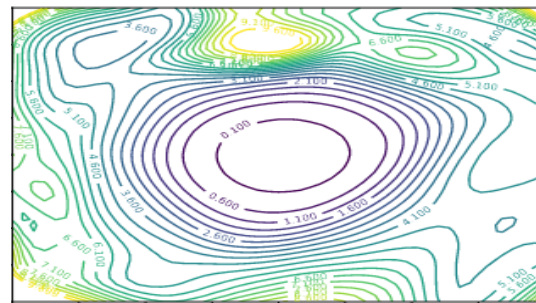
Since Resnets (2015) ...



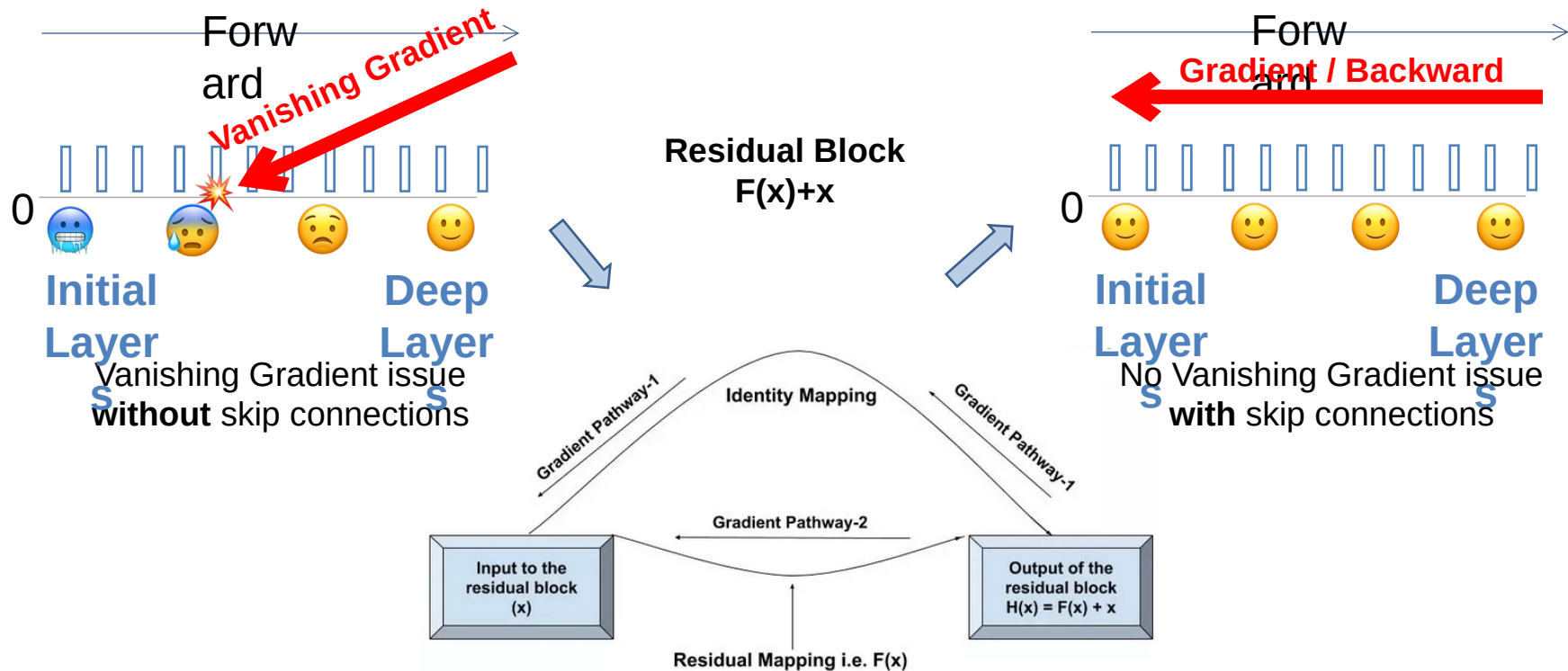
(a) without skip connections



(b) with skip connections

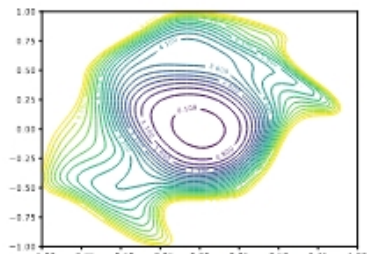


Residual Learning

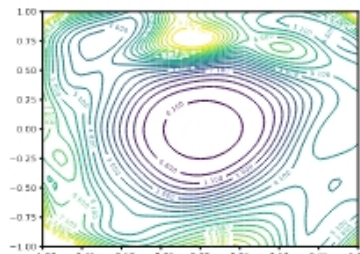


Residual Learning – depth impact

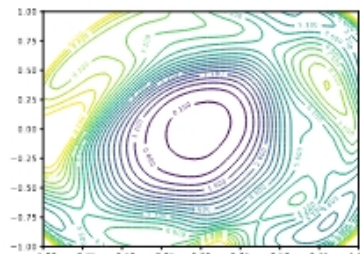
ResNet



(a) ResNet-20, 7.37%

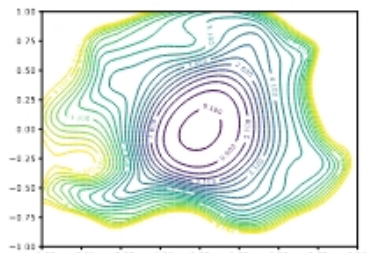


(b) ResNet-56, 5.89%

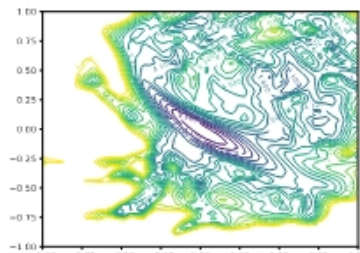


(c) ResNet-110, 5.79%

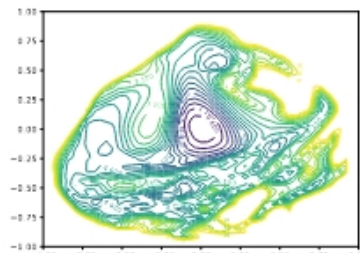
ResNet -
No Short
without skip connections



(d) ResNet-20-NS, 8.18%



(e) ResNet-56-NS, 13.31%



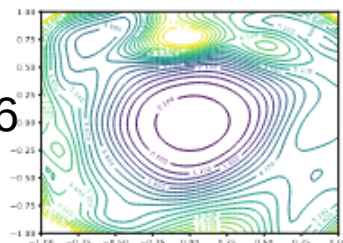
(f) ResNet-110-NS, 16.44%



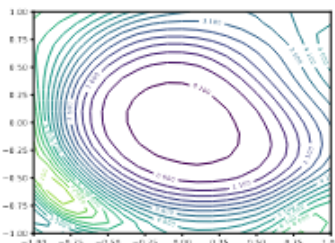
Residual Learning – width impact



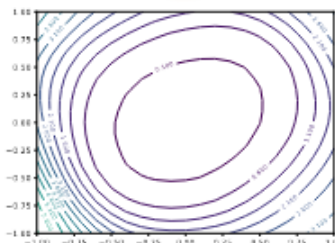
Wide-ResNet-56



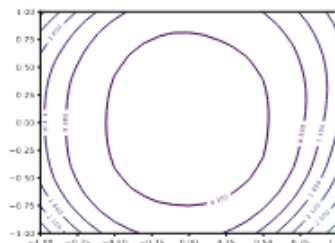
(a) $k = 1$, 5.89%



(b) $k = 2$, 5.07%

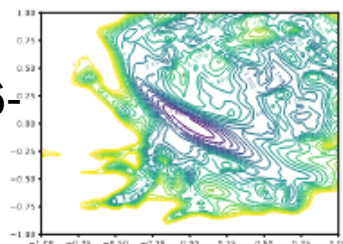


(c) $k = 4$, 4.34%

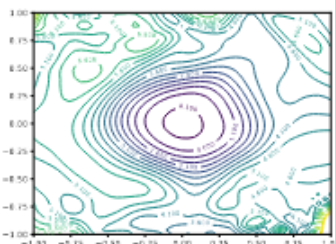


(d) $k = 8$, 3.93%

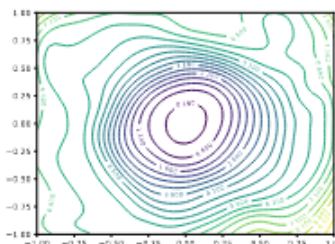
Wide-ResNet-56-
No Short
without skip connections



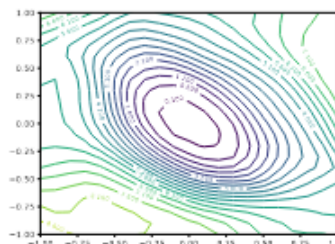
(e) $k = 1$, 13.31%



(f) $k = 2$, 10.26%



(g) $k = 4$, 9.69%



(h) $k = 8$, 8.70%



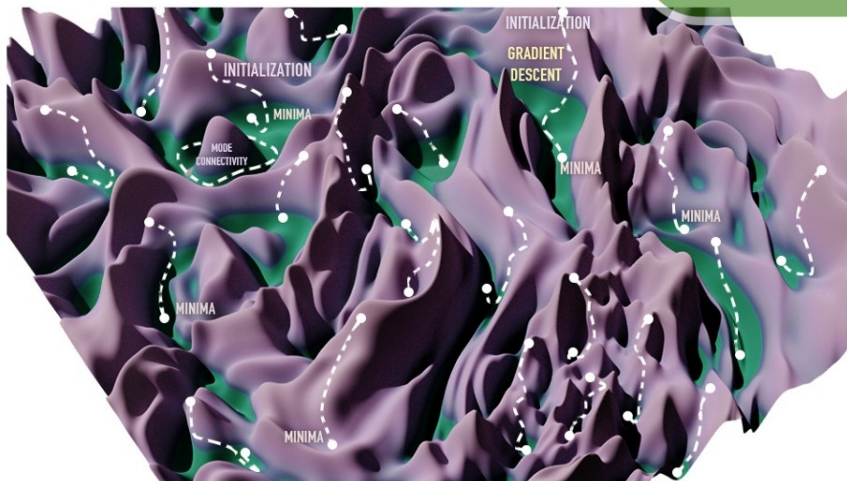
k = channel width coefficient compared to ResNet

Model Parameters Initialization

The Blessing of Dimensionality :

Local

NEARBY PATHS
TO CONVERGENCE



FINDING A MINIMA BECOMES A “LOCAL” CHALLENGE



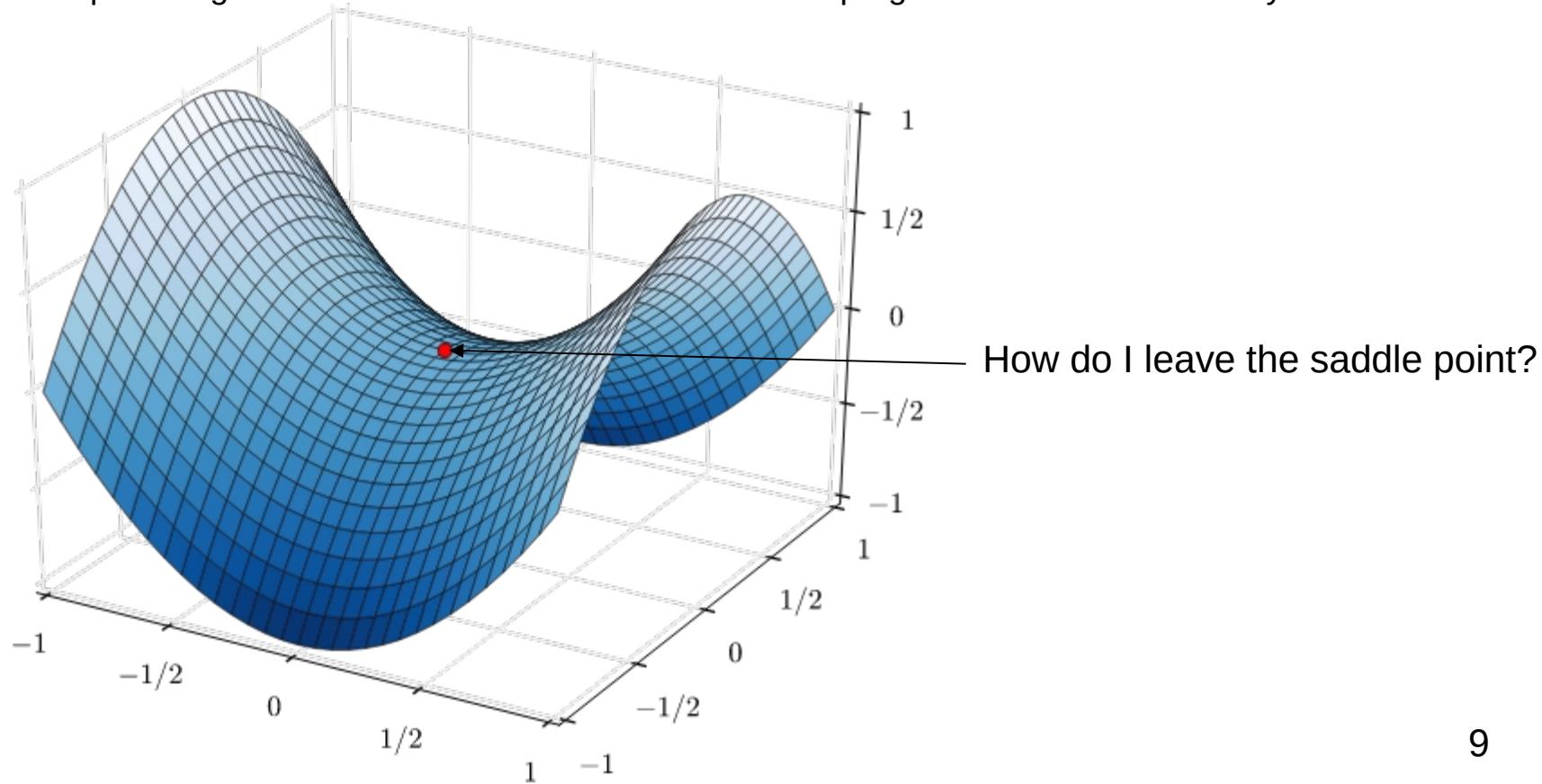
- Xavier Initialization
 - uniform
 - normal
- Kaiming Initialization
 - uniform
 - normal

By default in PyTorch:

- Best initialization algorithm depending on the type of layer (linear, convolutional, transform, ...).
- Today, it is no longer necessary to try to optimize initialization.

Saddle point Problem

Saddle point: A gradient close to zero which will make the progression of the model very slow



Learning Rate Scheduler

Learning Rate scheduler ◀

Cyclic scheduler ◀

One Cycle scheduler ◀

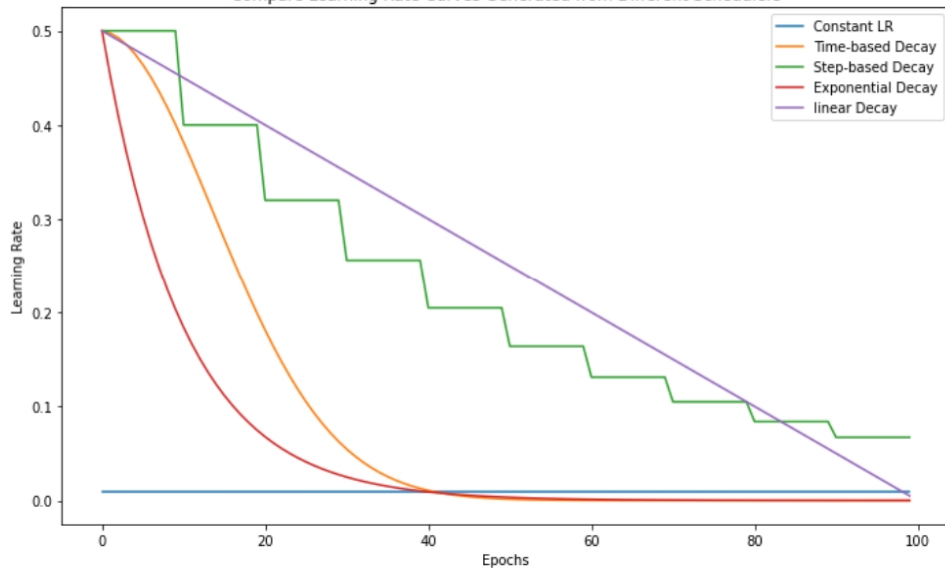
ScheduleFree ◀

LR Finder◀

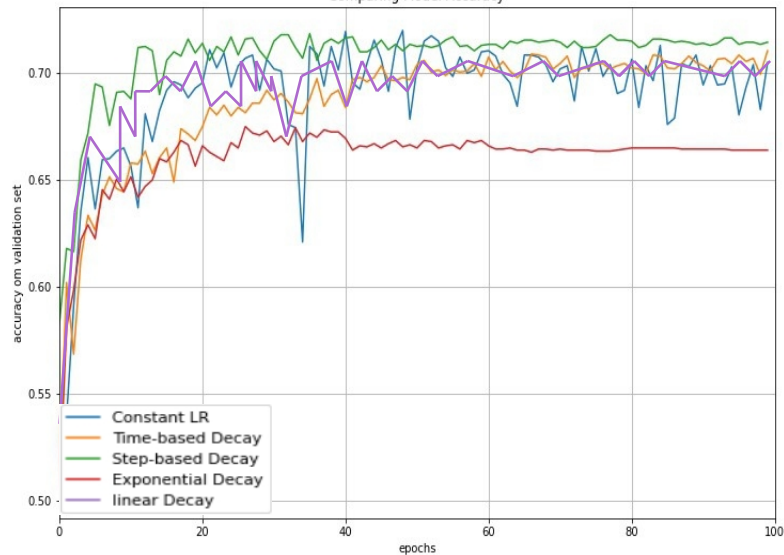
Learning Rate Scheduler

Learning rate decay

Compare Learning Rate Curves Generated from Different Schedulers



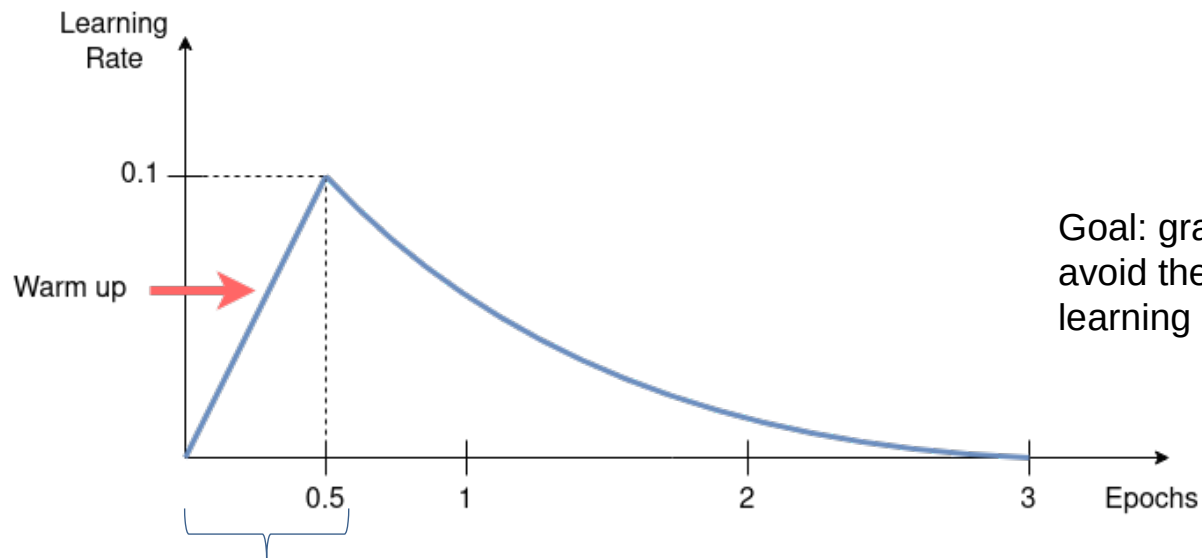
Comparing Model Accuracy



Learning Rate Scheduler

WARMUP for *large batches*

Problems: The first iterations have too much effect on the model (significant losses, high gradients, bias, etc.), a high learning rate can cause strong instability or divergence

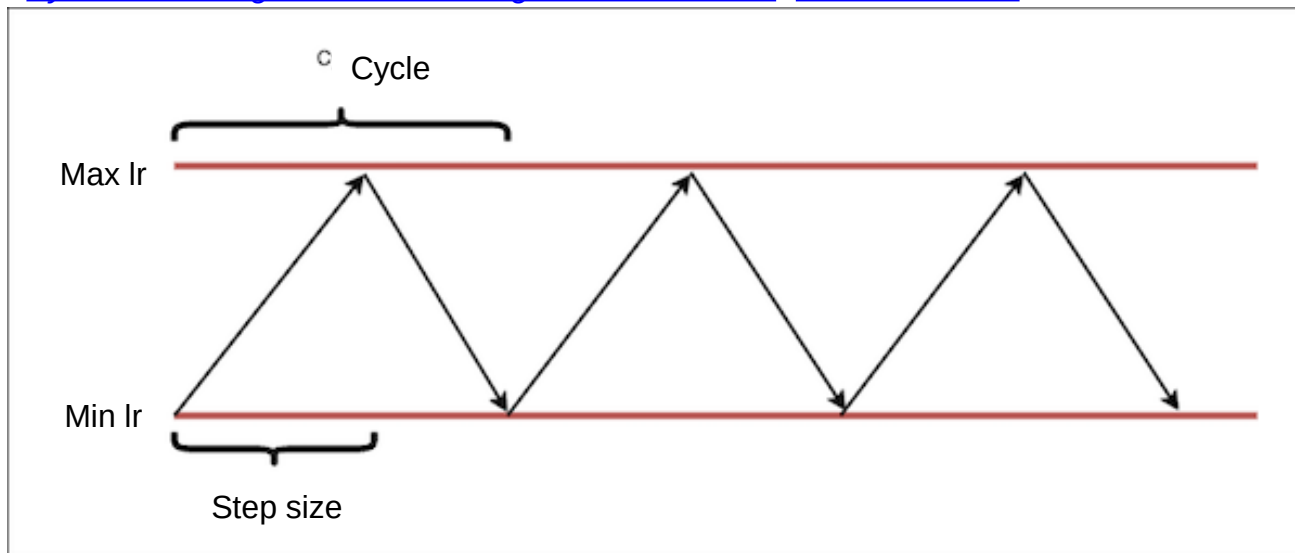


Goal: gradually increase the learning rate to avoid the risk of divergence at the start of learning

Untrusted Gradients

Cyclic Learning Rate Scheduler

[Cyclical Learning Rates for Training Neural Networks - Leslie N. Smith 2017](#)

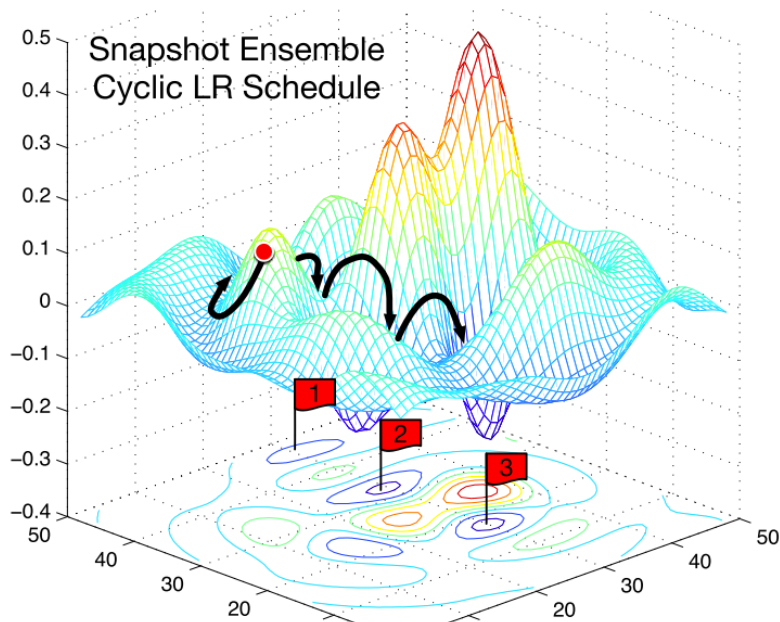
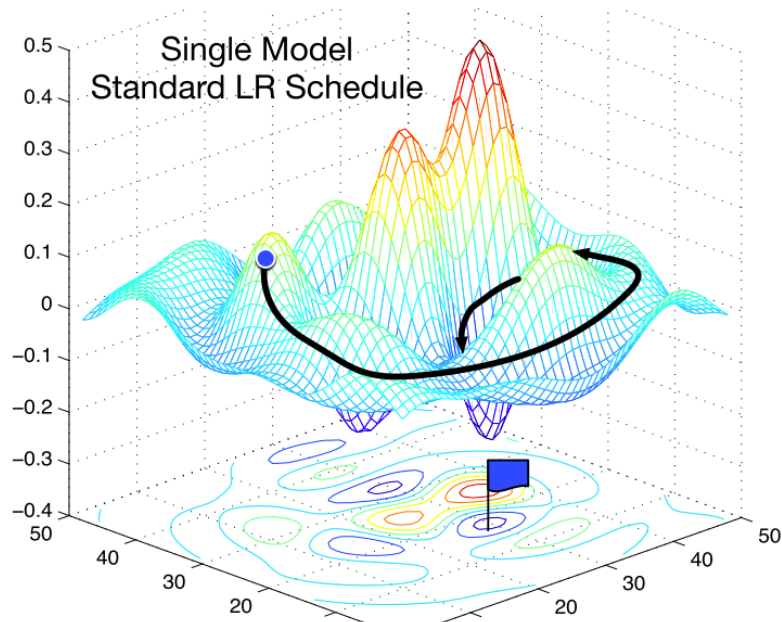


Paramètres :

- $\text{Step_size} = x * \text{epoch}$ ($2 \leq x \leq 10$)
- $\text{Base_lr} \rightarrow$ min convergence value
- $\text{max_lr} \rightarrow$ max convergence value

Succession of warmups and learning rate decays

Cyclic Learning Rate Scheduler

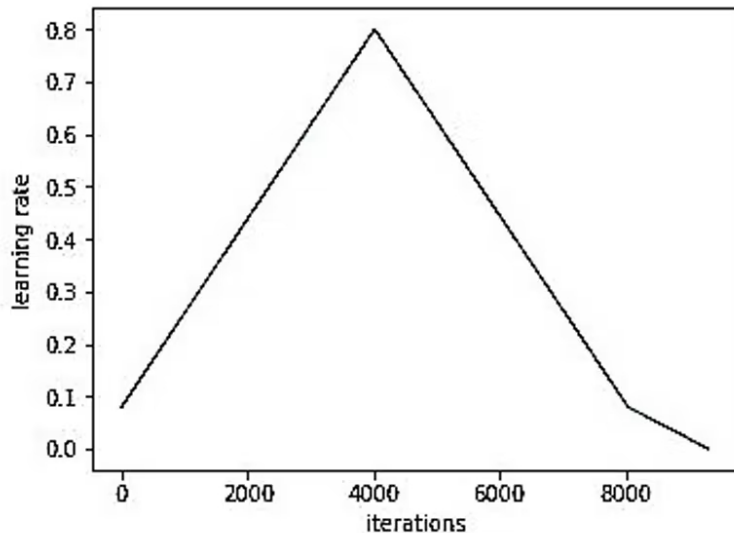


SNAPSHOT ENSEMBLES: TRAIN 1, GET M FOR FREE
Gao Huang, Yixuan Li, Geoff Pleiss

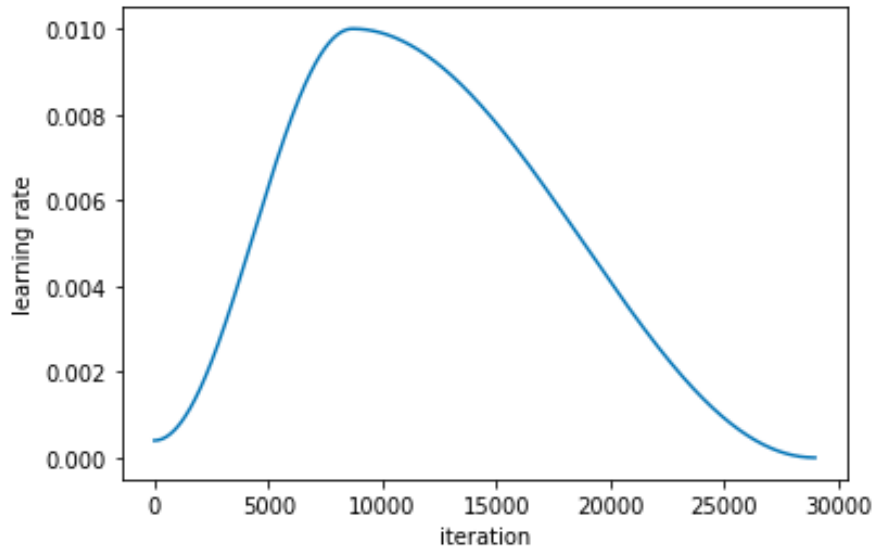
One Cycle Learning Rate

One cycle is enough! A disciplined approach to neural network hyper-parameters -
Leslie N. Smith 2018

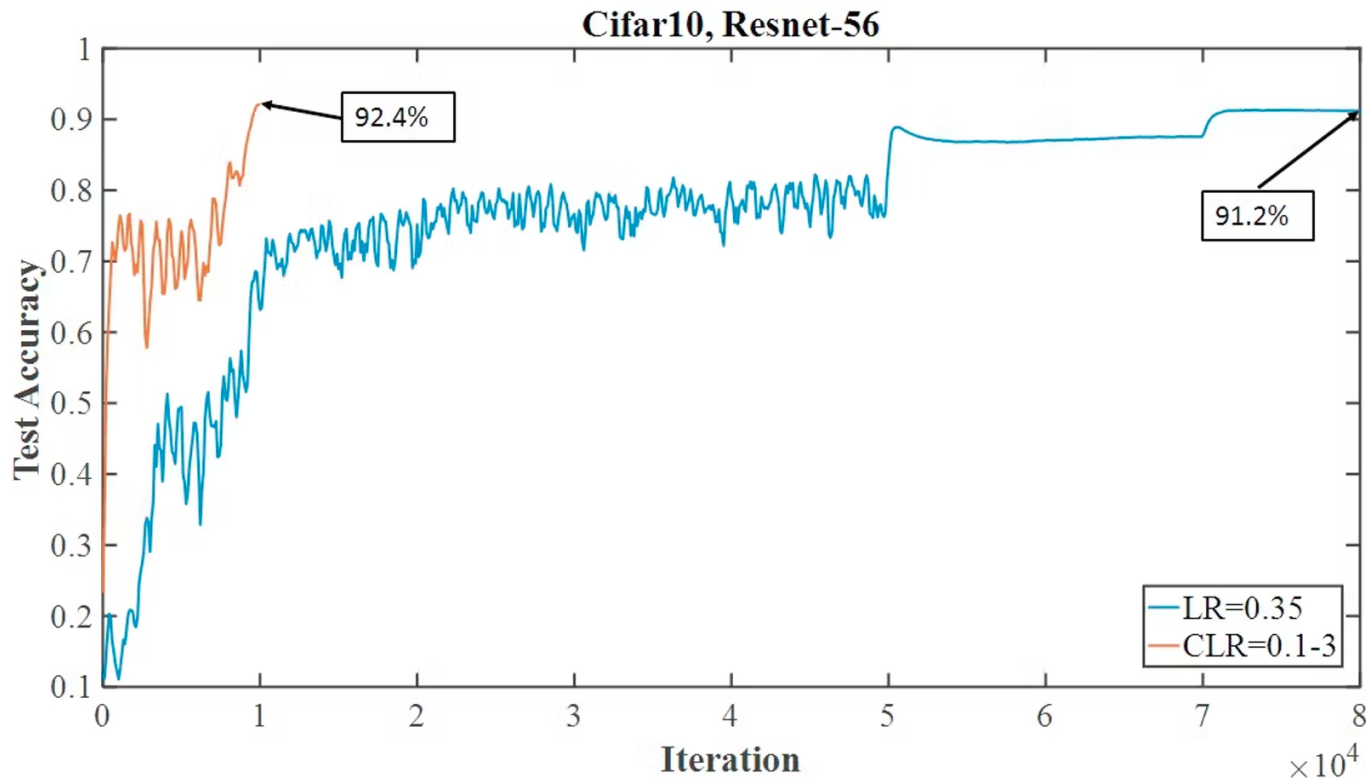
Proposition initiale



cosine annealing : Recommandation par FastAI



One Cycle Learning Rate - *Super convergence*



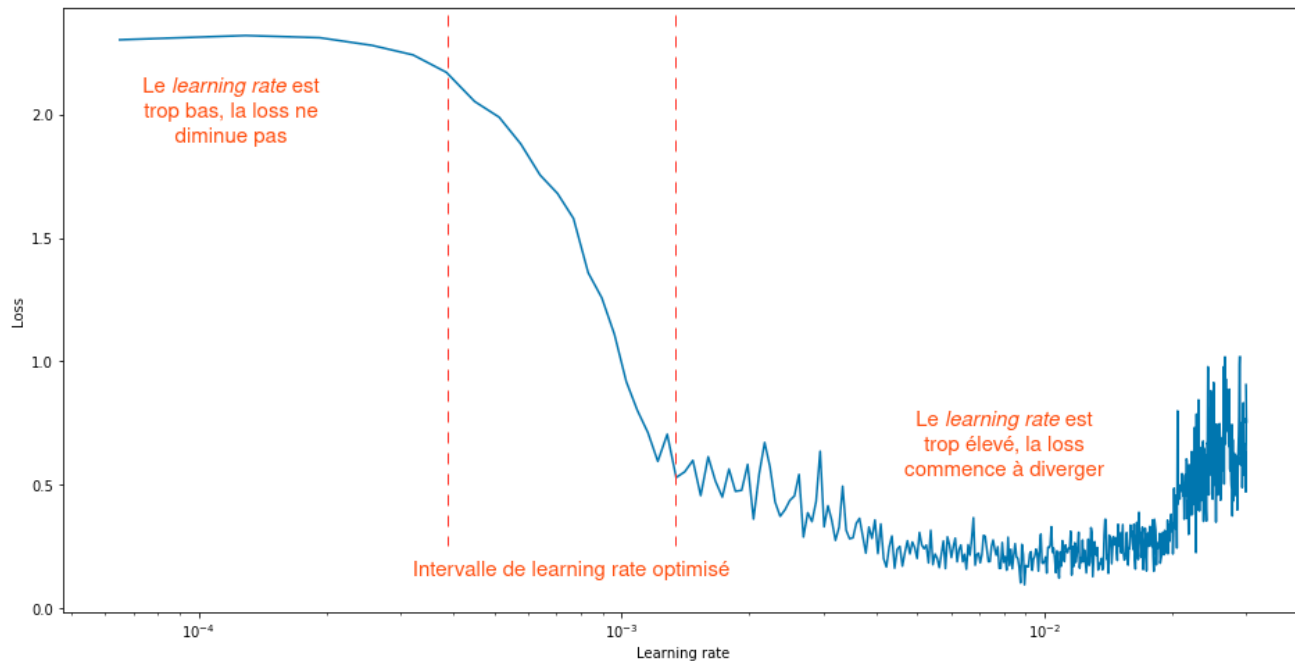
Faster convergence for equivalent final precision

Learning Rate Finder

Goal: Find the **optimal learning rate** values for your model, particularly for **the maximum value** of a *cyclic scheduler*

Run your model over a few epochs by increasing its learning rate

- Start of loss reduction
→ Minimal learning rate
- Start of loss variation
→ Maximum learning rate



Learning Rate Scheduler

Each **scheduler** has *its own settings*

```
import torch.optim as opt
```

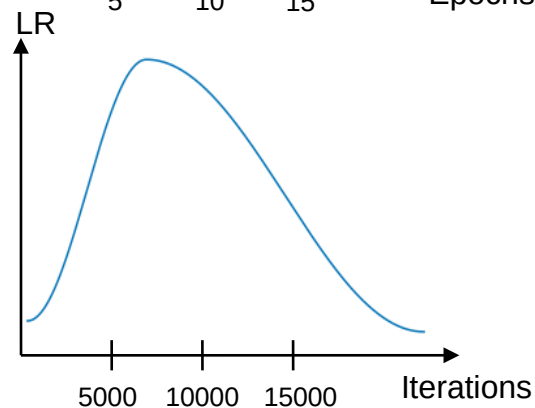
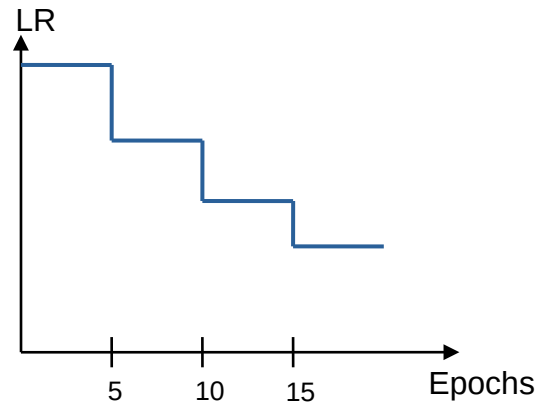
```
scheduler = opt.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
```

```
for epoch in range(100):  
    train(...)  
    validate(...)  
    scheduler.step()
```

```
import torch.optim as opt
```

```
scheduler = opt.lr_scheduler.CyclicLR(optimizer, base_lr=0.01, max_lr=0.1)
```

```
for epoch in range(10):  
    for batch in data_loader:  
        train_batch(...)  
        scheduler.step()
```



Gradient Descent Optimizer

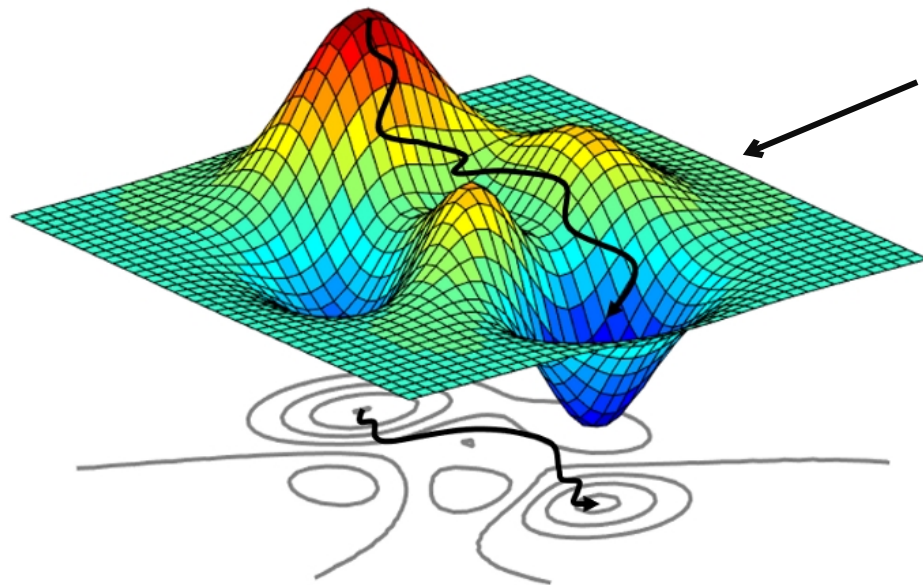
SGD ◀

ADAM ◀

ADAMW ◀

Optimizer - SGD

The **optimizer** is the algorithm that **controls the gradient descent** and the **minimum search** with the aim of optimizing the learning time and the final metric.



SGD = Stochastic Gradient Descent
*Calculating the Gradient and updating the weights
at each batch*

- + Batch size and learning rate adaptable according to conflicting needs:
 - Exploration to find the best local minimum
 - Acceleration of gradient descent

SGD with Momentum

$$m_0 = 0$$

Momentum coefficient

$$m_i = \boxed{\beta} * m_{i-1} + (1 - \beta) * g_i$$

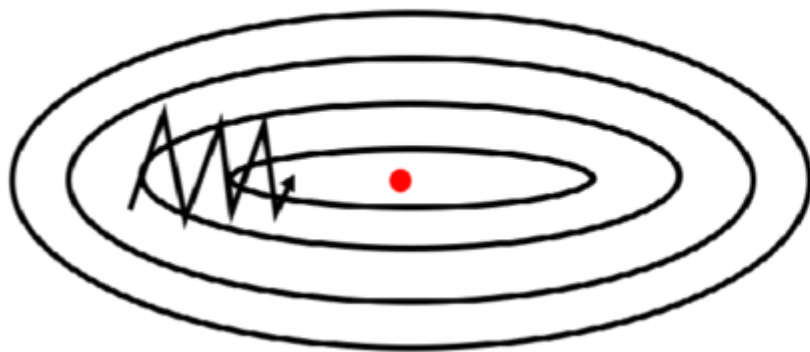
$$\theta_i = \theta_{i-1} - \alpha * m_i$$

Goal: Take **previous gradients** into consideration for **faster** gradient descent.

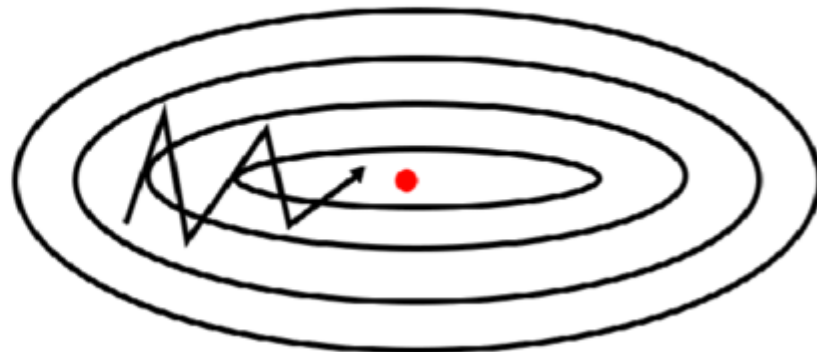
Recommended initial value: 0.9

$$0.85 < \beta < 0.95$$

SGD without momentum



SGD with momentum

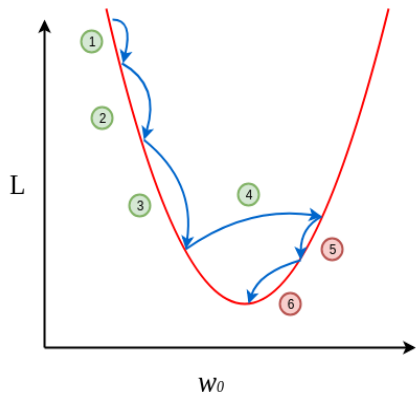


+ Allows you to **converge more quickly**

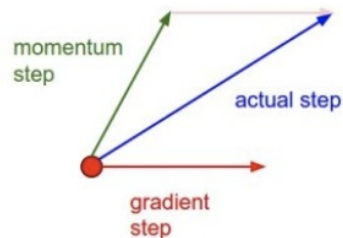
- **No guarantee** that momentum will take us in the right direction

Momentum type

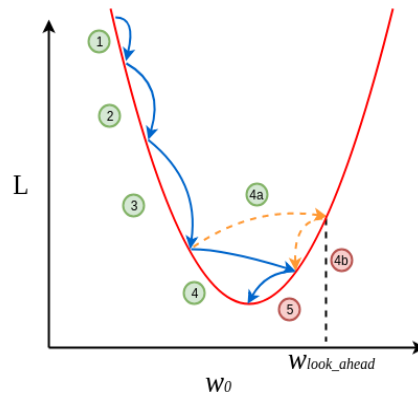
Momentum



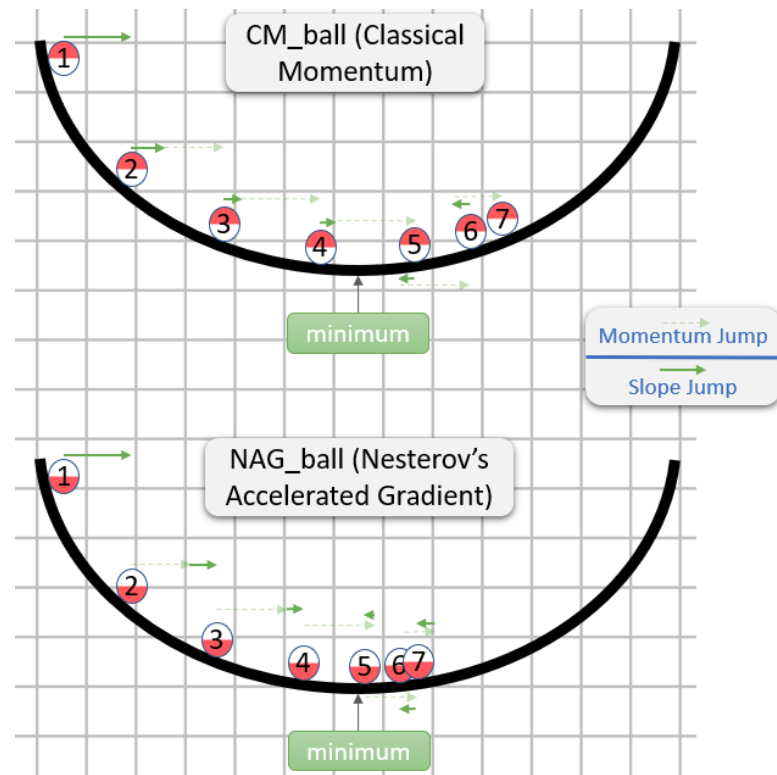
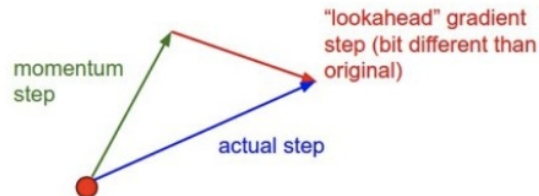
Momentum update



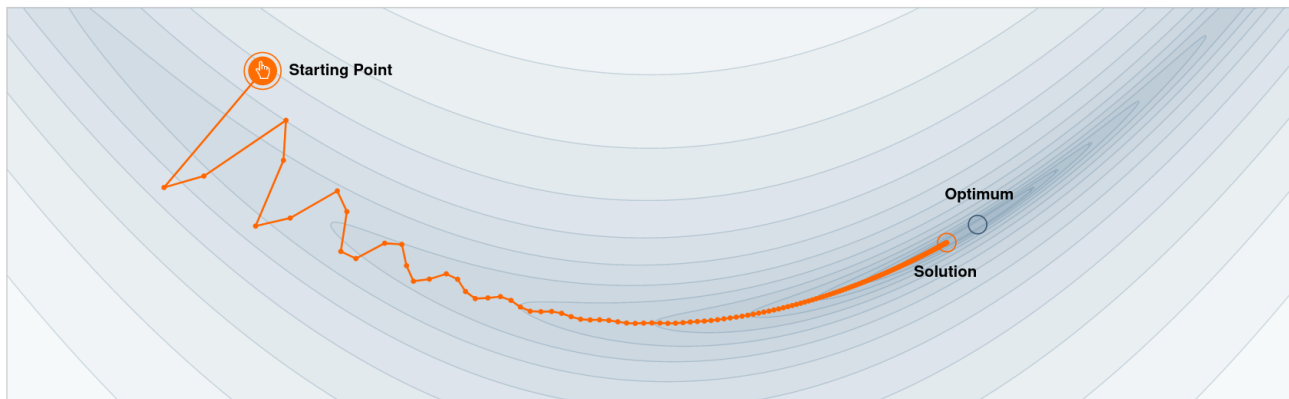
Nesterov momentum



Nesterov momentum update



Why Momentum Works ?



Step-size $\alpha = 0.02$



Momentum $\beta = 0.99$



We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

GABRIEL GOH
UC Davis

April. 4
2017

Citation:
Goh, 2017

<https://distill.pub/2017/momentum/>

Adaptive Optimizers

Rather than controlling the gradient descent manually with the learning rate...

... We can adapt the *learning rate* **for each weight** of the model according to the **gradient**, the **gradient²**, or the **norm of the weights** of the layer!!!

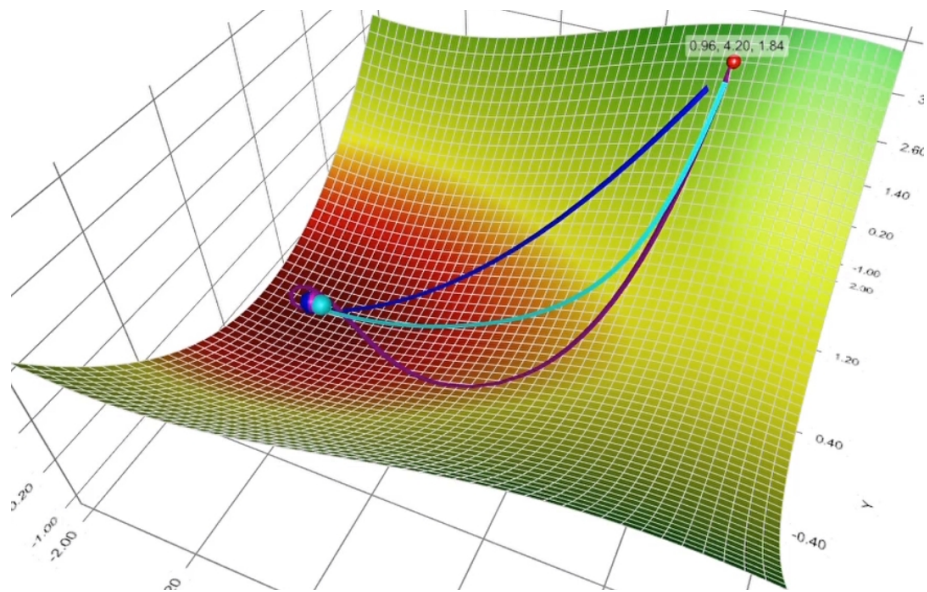
Examples :

- AdaGrad,
- AdaDelta,
- RMSprop
- Adam

Specialized for larges batches :

- LARS
- LAMB

- SGD (no *momentum*)
- SGD (with *momentum*)
- Adam



Adam

$$m_i = \beta_1 * m_{i-1} + (1 - \beta_1) * g_i$$

$$v_i = \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}$$

Correction of biases of the first iterations

$$\hat{v}_i = \frac{v_i}{1 - \beta_2^i}$$

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i + \epsilon}} * \hat{m}_i$$

Parameters:

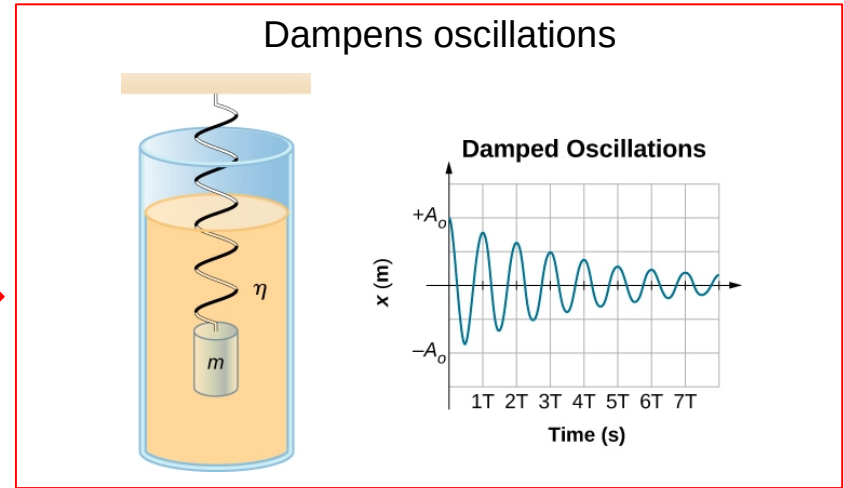
β_1 & β_2 = Regression rate ($\beta_1 = 0.9$ & $\beta_2 = 0.999$)

ϵ = Very small value to avoid division by zero

Adam : Adaptative moment estimation

First moment : sliding mean

Second moment : sliding non-centered variance

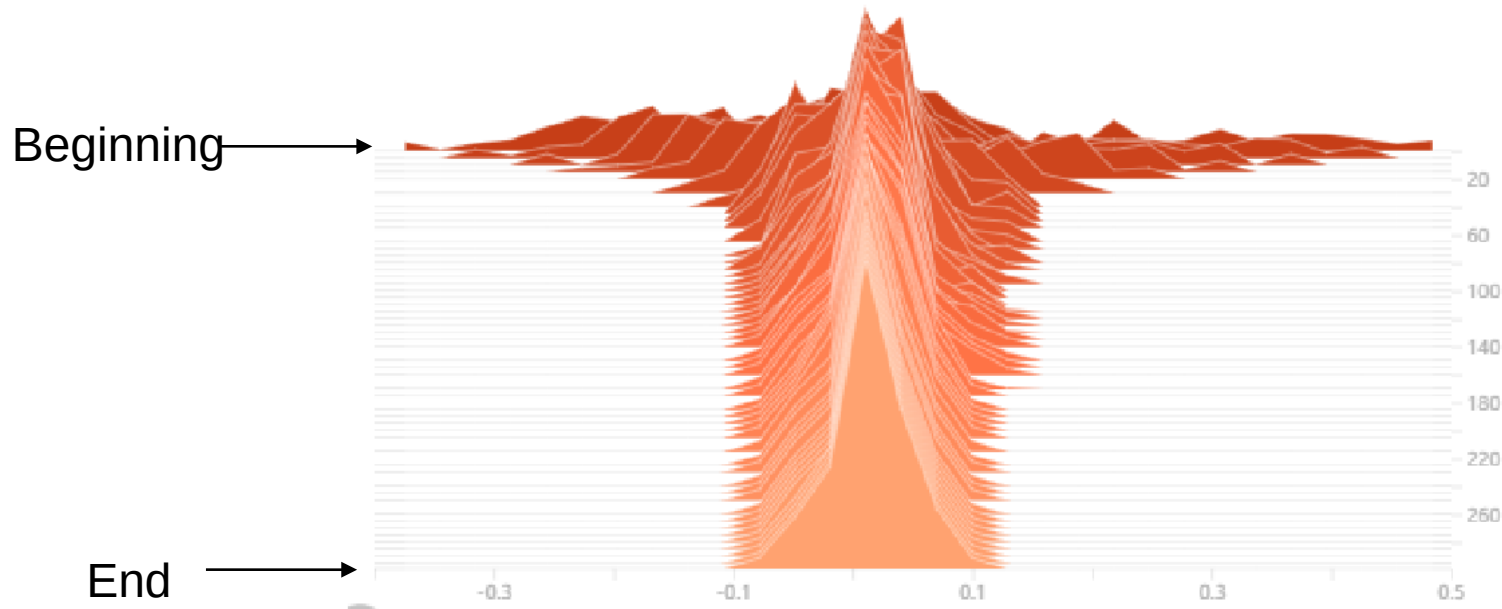


Goal: **Adapt the importance of weight** updates based
on
previous gradients and gradient variability.

Weight decay

A neural network that **converges and generalizes correctly*** generally has weights **that tend to 0**. *(neither underfitting nor overfitting)

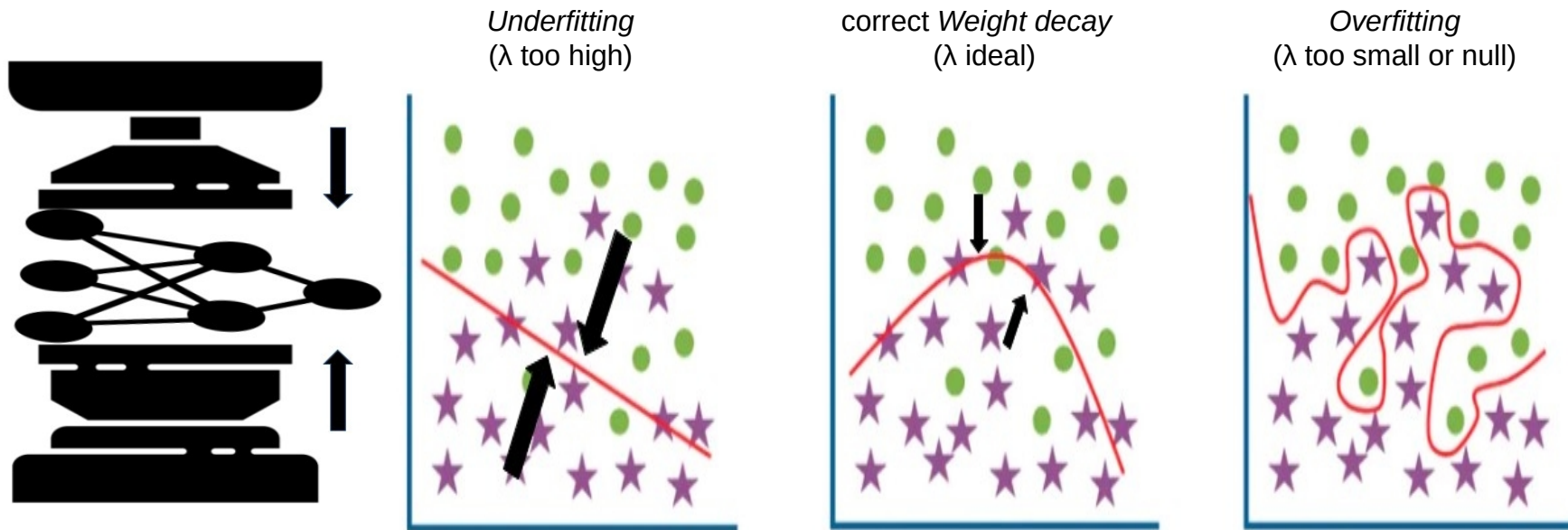
Distribution of weights during learning:



Weight decay

Preferable to standard L2 regularization defined in loss function

λ : weight decay parameter (between 0 and 0.1)



The weight decay technique, defined in the optimizer, makes it possible to force the weights to converge towards values close to zero.

Weight decay and decoupled weight decay

ADAM

For $i = 1$ to ...

$$\begin{aligned} g_i &= \nabla_{\theta} f_i(\theta_{i-1}) + \lambda \theta_{i-1} \\ m_i &= \beta_1 * m_{i-1} + (1 - \beta_1) * g_i \\ v_i &= \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2 \\ \hat{m}_i &= \frac{m_i}{1 - \beta_1^i} \\ \hat{v}_i &= \frac{v_i}{1 - \beta_2^i} \\ \theta_i &= \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i} + \epsilon} * \hat{m}_i \end{aligned}$$

Return θ_i

Weight decay

ADAMW

For $i = 1$ to ...

$$\begin{aligned} g_i &= \nabla_{\theta} f_i(\theta_{i-1}) \\ m_i &= \beta_1 * m_{i-1} + (1 - \beta_1) * g_i \\ v_i &= \beta_2 * v_{i-1} + (1 - \beta_2) * g_i^2 \\ \hat{m}_i &= \frac{m_i}{1 - \beta_1^i} \\ \hat{v}_i &= \frac{v_i}{1 - \beta_2^i} \\ \theta_i &= \theta_{i-1} - \frac{\alpha}{\sqrt{\hat{v}_i} + \epsilon} * \hat{m}_i - \alpha \lambda \theta_{i-1} \end{aligned}$$

Return θ_i

Decoupled weight decay

Evolution of weight decay:

Decoupled weight decay (decoupled from momentum!!)

- SGD and Adam with weight decay
- SGD and AdamW with decoupled weight decay

SGD and SGD are roughly equivalent in performance.

However **AdamW is noticeably better than Adam!!**

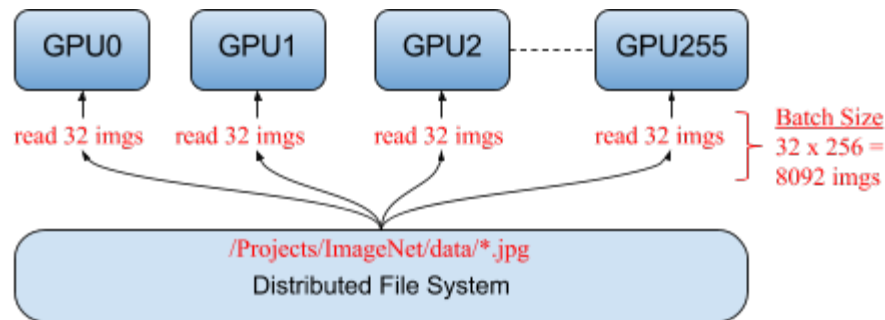
Optimization of large batches

Large batches issues ◀

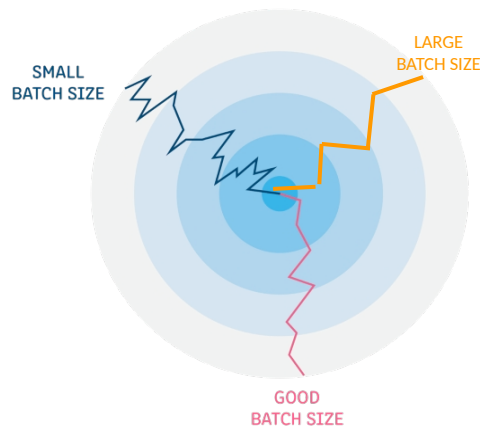
Learning Rate Scaling & Batch Schedulers ◀

Large batches optimizers ◀

Large Batches with Data Parallelism



Data Parallelism: This parallelism generates very large batches



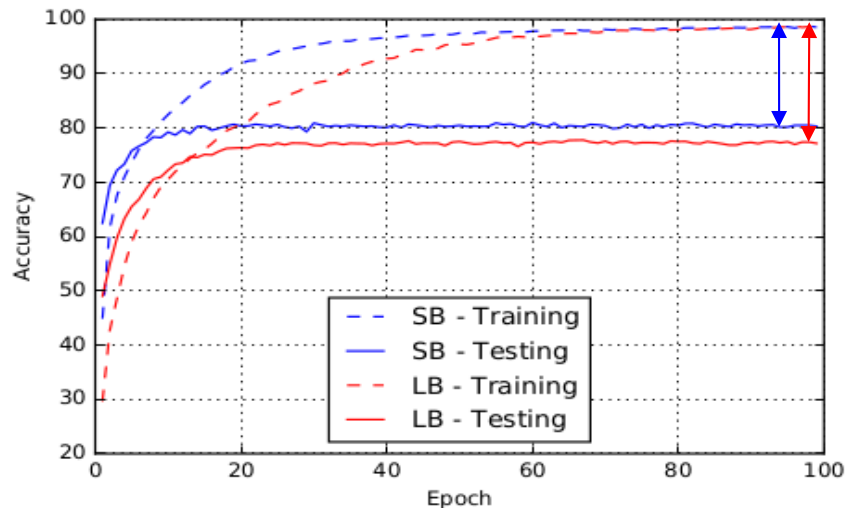
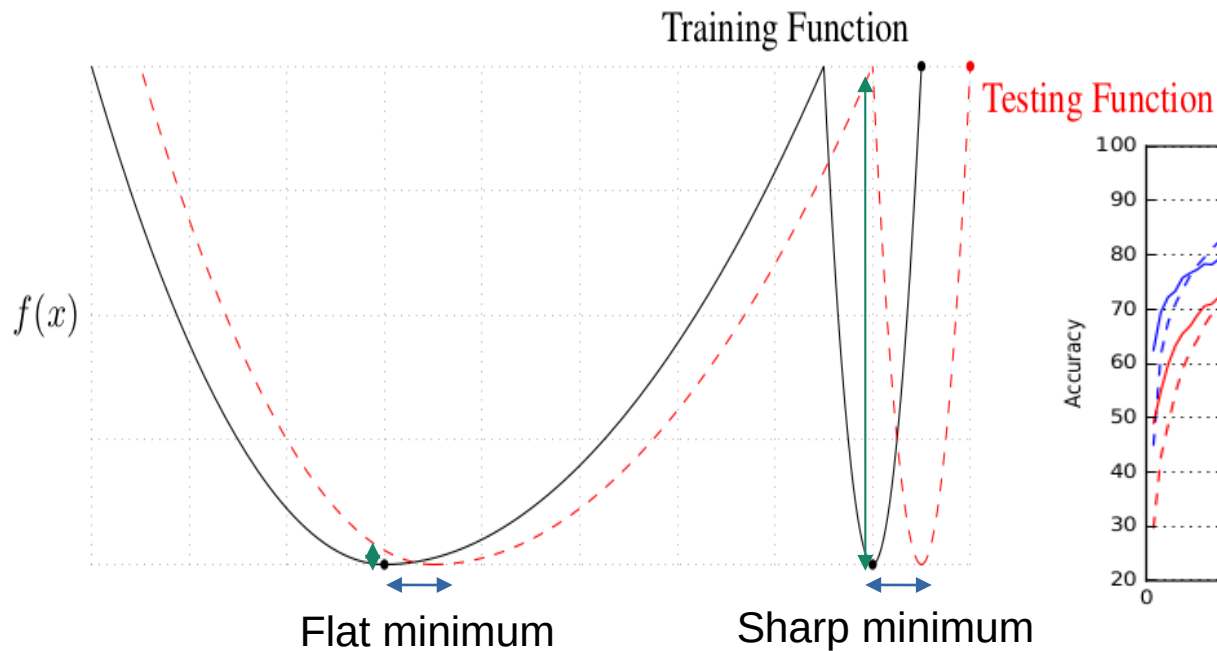
Problem: *Batch* that are too large (> 512) tend to result in **poorer performance**



Large Batches

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, Ping Tak Peter Tang



Comparison of training a convolutional network with small batch (SB) and large batch (LB) on CIFAR 10

The *larger the batch*, the more the model tends to converge towards **steep and narrow minima**.

Large Batches : Learning rate scaling

When the *size of the global batch* is **considerably increased**, it is often *necessary* to **scale the learning rate**:

N = Number of parallel processes

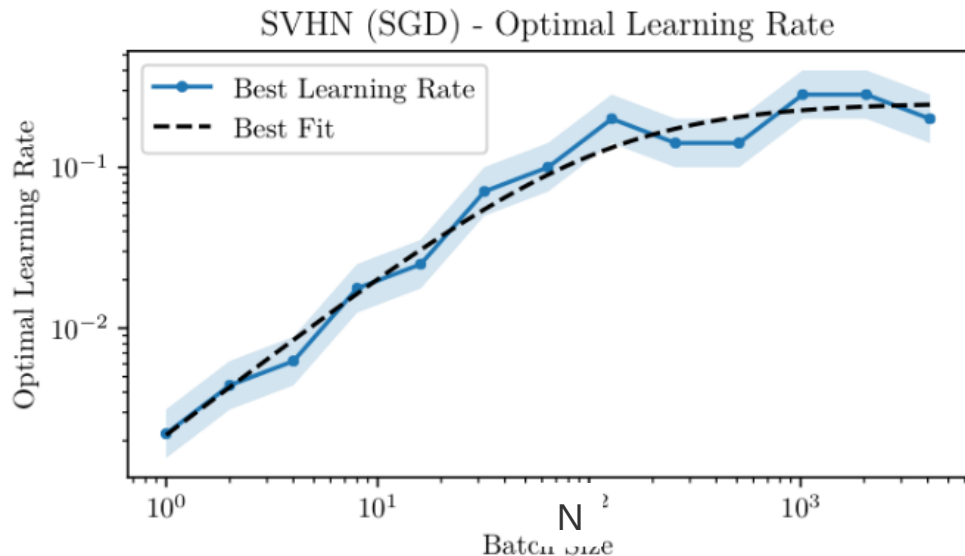
Linear growth of *learning rate*:

$$\alpha \rightarrow N * \alpha$$

Square root growth of *learning rate*:

$$\alpha \rightarrow \sqrt{N} * \alpha$$

Optimal: **linear growth** at first then **square root**
(recommended by OpenAI)



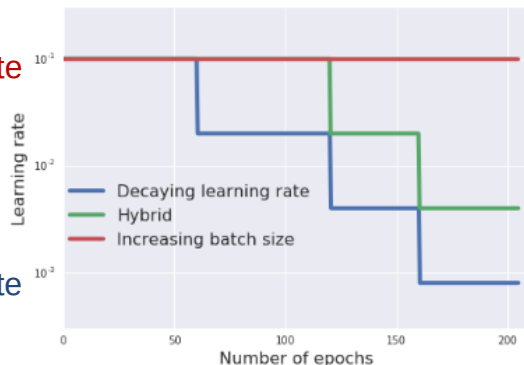
An Empirical Model of Large-Batch Training
Sam McCandlish, Jared Kaplan, Dario Amodei

Batch Size Scheduler

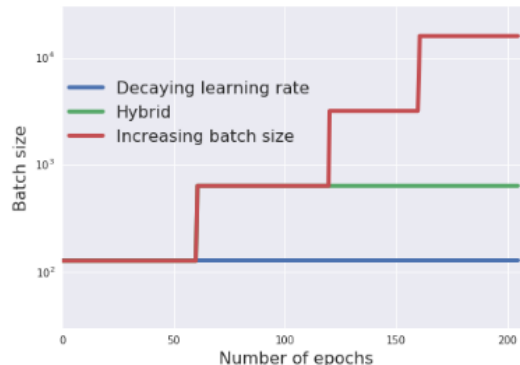
=> Alternative to Learning Rate Scheduler

DON'T DECAY THE LEARNING RATE, INCREASE THE BATCH SIZE

High Learning Rate

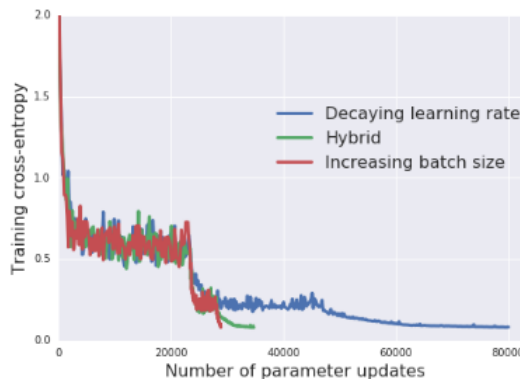
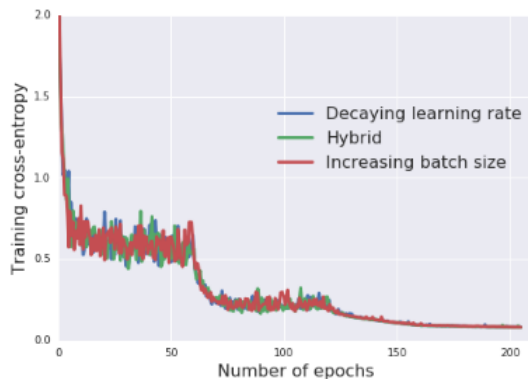


Low Learning Rate



Large Batch

Small Batch



Large Batches

Trends :

Flat Minimum | Sharp Minimum

- Test Loss | + Test Loss

Slow Descent | Fast Descent

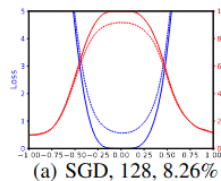
← Small Batch → Large Batch

SGD ADAM

← Weight Decay → w/o W.Decay

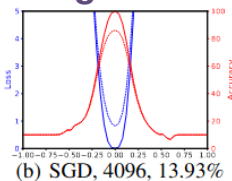
SGD

Small Batch

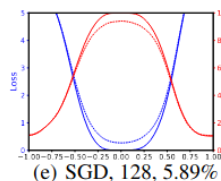


(a) SGD, 128, 8.26%

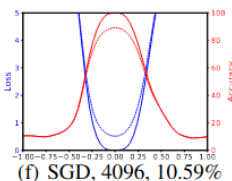
Large Batch



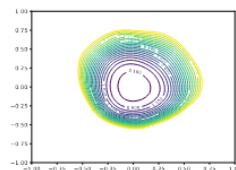
(b) SGD, 4096, 13.93%



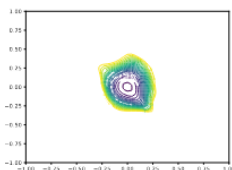
(e) SGD, 128, 5.89%



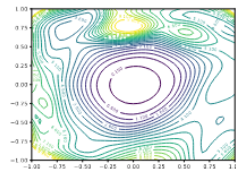
(f) SGD, 4096, 10.59%



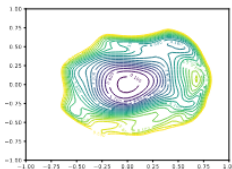
(i) SGD, 128, 8.26%



(j) SGD, 4096, 13.93%



(m) SGD, 128, 5.89%



(n) SGD, 4096, 10.59%

Weight
Decay

= 0

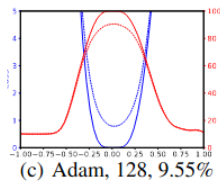
= $5e^{-4}$

= 0

= $5e^{-4}$

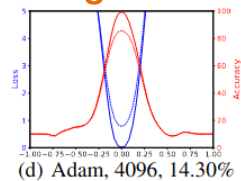
Adam

Small Batch

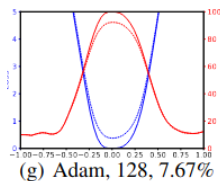


(c) Adam, 128, 9.55%

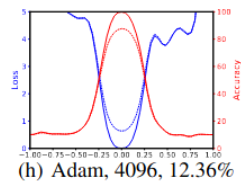
Large Batch



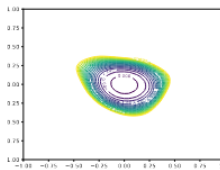
(d) Adam, 4096, 14.30%



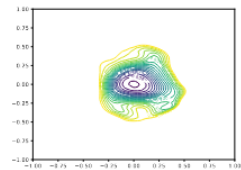
(g) Adam, 128, 7.67%



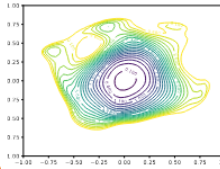
(h) Adam, 4096, 12.36%



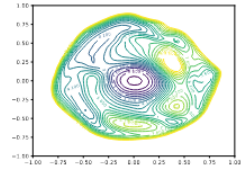
(k) Adam, 128, 9.55%



(l) Adam, 4096, 14.30%



(o) Adam, 128, 7.67%



(p) Adam, 4096, 12.36%

Large Batches Optimisers - LARS

LARS = “Layer-wise Adaptive Rate Scaling.”

Adaptation of SGD with momentum with the addition of a **trust ratio** for **each layer** which depends on the evolution of the layer's gradient

r = Trust ratio

l = layer number

$$m_i = \beta * m_{i-1} + (1 - \beta) * (g_i + \frac{\lambda \theta_{i-1}}{r})$$

$$r_1 = \left\| \theta_{i-1}^l \right\|_2$$

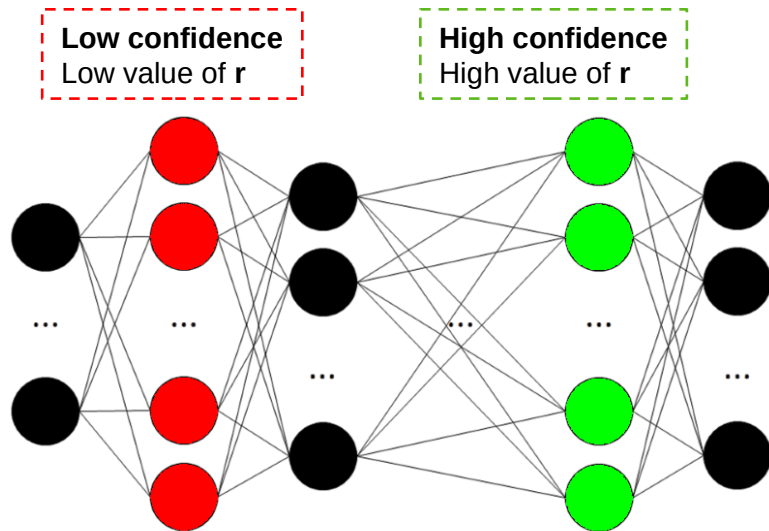
$$r_2 = \left\| m_i^l \right\|_2$$

$$r = r_1 / r_2$$

$$\alpha^l = r * \alpha$$

Weight decay

$$\theta_i^l = \theta_{i-1}^l - \alpha^l * m_i^l$$



Goal: Adapt the importance of weight updates based on a **trust ratio** calculated for each layer of the network.

Large Batches Optimisers - LAMB

LAMB pour “Layer-wise Adaptive Moments optimizer for Batch training.”

Adaptation of ADAM with momentum with the addition of a **trust ratio** for **each layer** which depends on the evolution of the layer's gradient

r = Trust ratio

l = layer number

$$r_1 = \| \theta_{i-1}^l \|_2$$

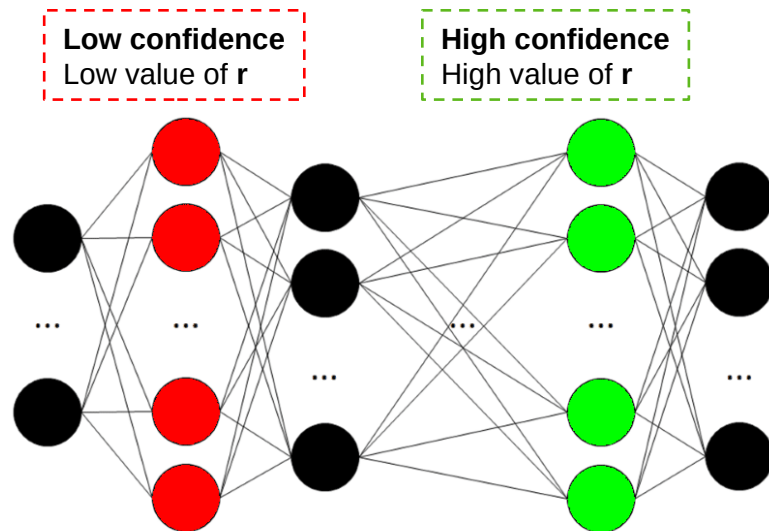
$$r_2 = \left\| \frac{\hat{m}_i^l}{\sqrt{\hat{v}_i^l + \epsilon}} + \lambda \theta_{i-1}^l \right\|_2$$

$$r = r_1 / r_2$$

$$\alpha^l = r * \alpha$$

$$\theta_i^l = \theta_{i-1}^l - \alpha^l * \left(\frac{\hat{m}_i^l}{\sqrt{\hat{v}_i^l + \epsilon}} + \lambda \theta_{i-1}^l \right)$$

Decoupled weight decay



Goal: Adapt the importance of weight updates based on a **trust ratio** calculated for each layer of the network.

Optimizer implementation

SGD

```
import torch.optim as opt
```

```
SGD_optimizer = opt.SGD(params, lr, momentum=0, weight_decay=0, nesterov=False, ...)
```

ADAMW

```
import torch.optim as opt
```

```
ADAM_optimizer = opt.AdamW(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0.05,...)
```

LAMB

```
from apex.optimizers import FusedLamb
```

```
LAMB_optimizer = FusedLamb(params, lr=0.001, betas=(0.9, 0.999), weight_decay=0,  
adam_w_mode=True)
```

LARC

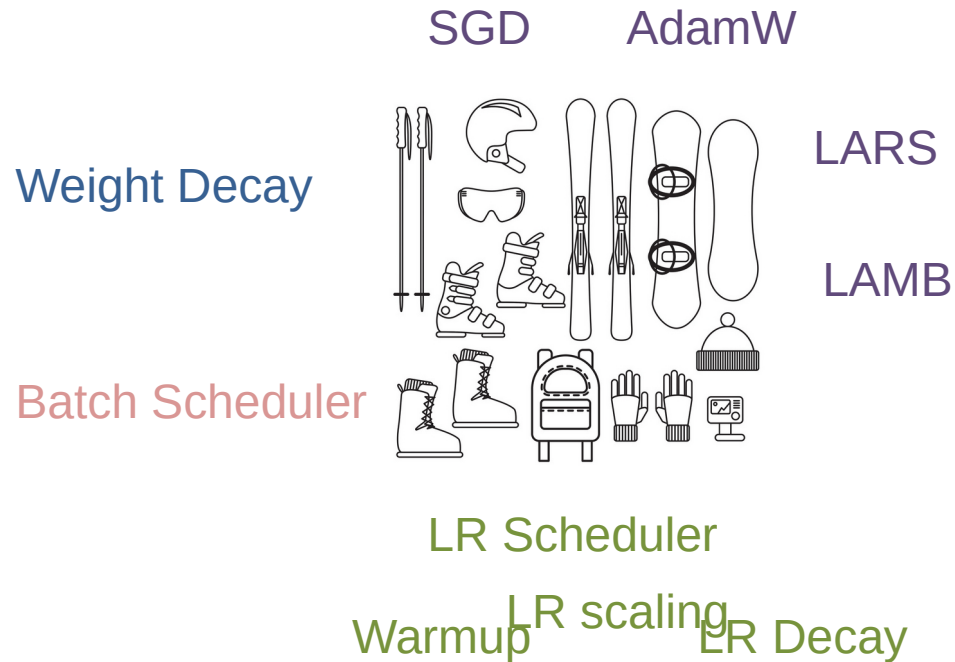
LARS
optimisation
from APEX

```
import torch.optim as opt
```

```
from apex.parallel.LARC import LARC
```

```
base_optimizer = opt.SGD(params, lr=0.001, momentum=0.9, weight_decay=0)  
optimizer = LARC(base_optimizer)  
scheduler = opt.lr_scheduler.CyclicLR(base_optimizer, base_lr=0.01, max_lr=0.1)
```

Large Batches Rider



BLOOM example

BLOOM
176B params 59 languages Open-access



95281 steps (116.8 days)

AdamW,

$\beta_1=0.9$, $\beta_2=0.95$, $\text{eps}=1\text{e-}8$

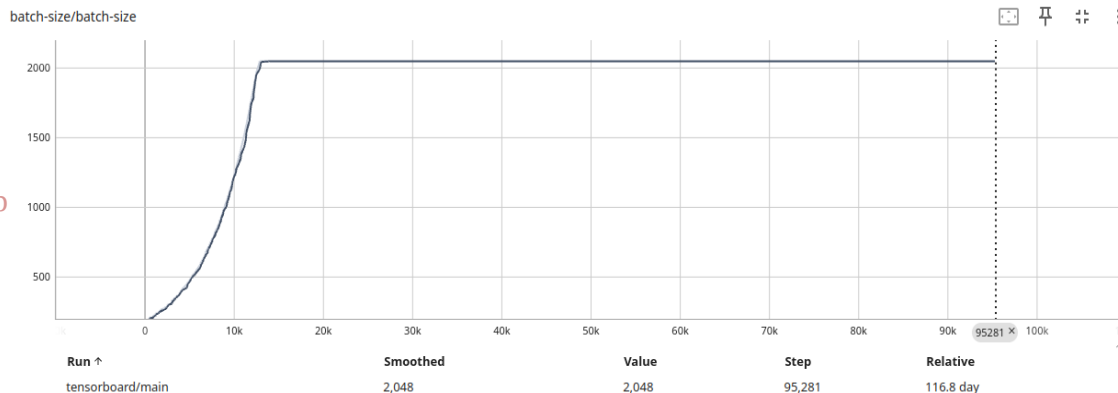
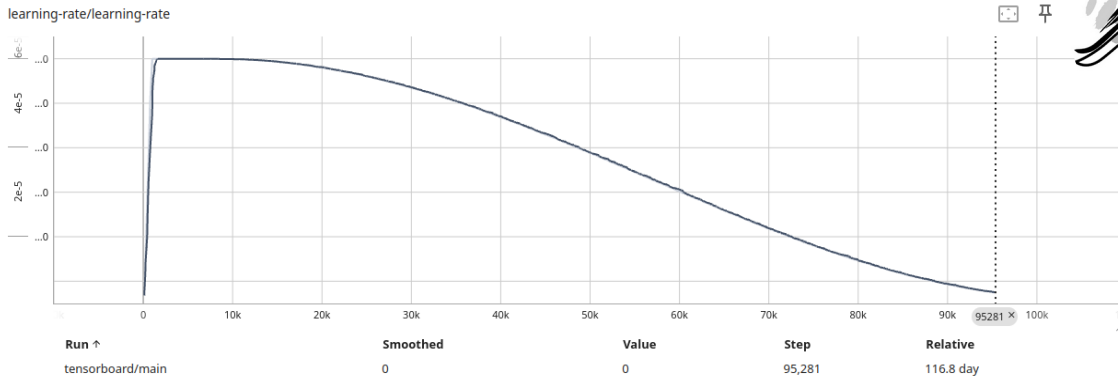
Weight Decay of 0.1

LR Scheduler

- peak= $6\text{e-}5$
- warmup over 375M tokens
- cosine decay for learning rate down to 10% of its value, over 410B tokens

Batch Scheduler

- start from 32k tokens (GBS=16)
- increase linearly to 4.2M tokens/step (GBS=2048) over $\sim 20\text{B}$ tokens
- then continue at 4.2M tokens/step (GBS=2048) for 430B tokens



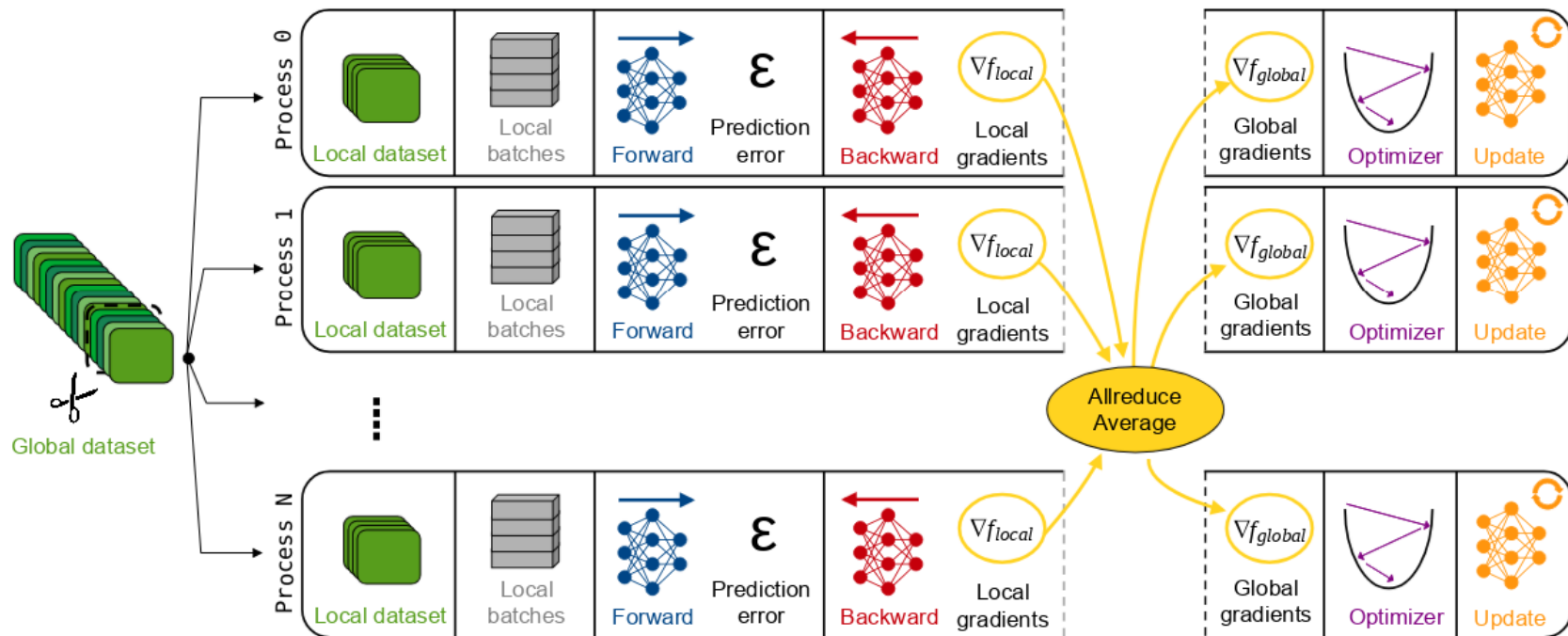
Reducing Optimizer Communication Costs

AllReduce Bottleneck ◀

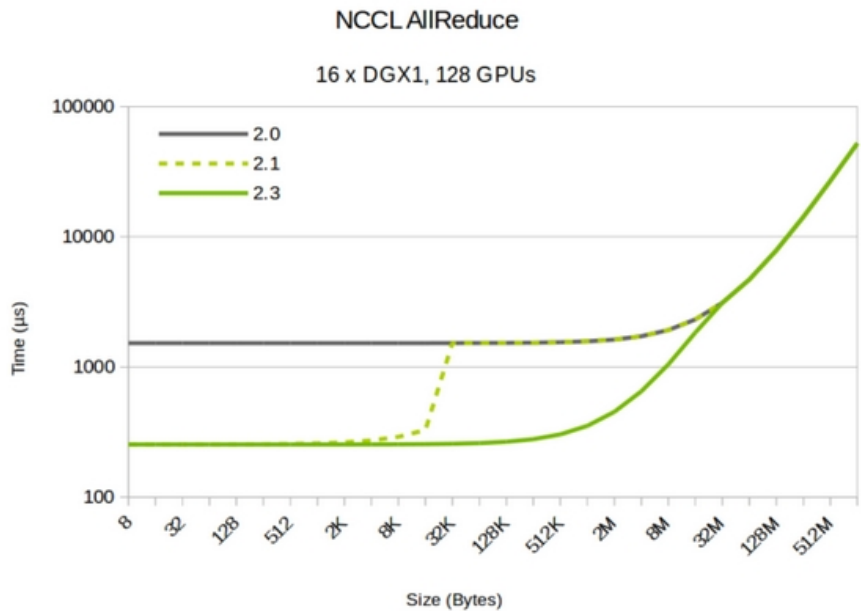
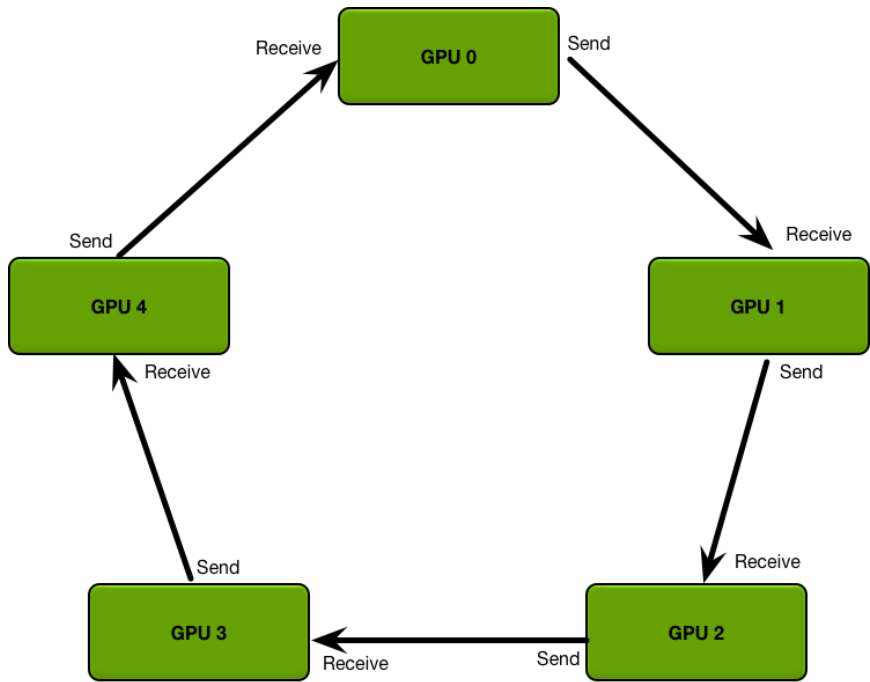
PowerSGD ◀

DiLoCo ◀

AllReduce bottleneck



DDP Communication Costs



Gradient compression - PowerSGD

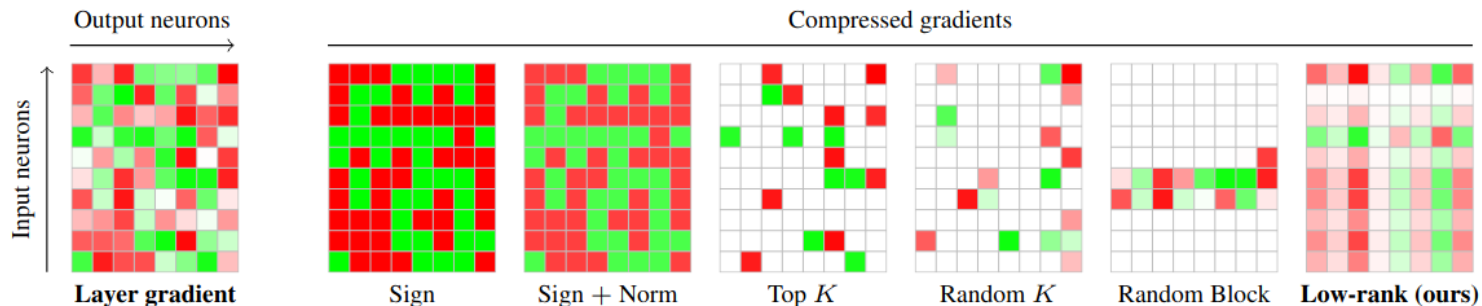


Figure 1: Compression schemes compared in this paper. Left: Interpretation of a layer’s gradient as a matrix. Coordinate values are color coded (positive, negative). Right: The output of various compression schemes on the same input. Implementation details are in Appendix G.

Table 3: POWERSGD with varying rank. With sufficient rank, POWERSGD accelerates training of a RESNET18 and an LSTM by reducing communication, achieving test quality on par with regular SGD in the same number of iterations. The time per batch includes the forward/backward pass (constant). See Section 5 for the experimental setup.

Image classification — RESNET18 on CIFAR10

Algorithm	Test accuracy	Data sent per epoch	Time per batch
SGD	94.3% —	1023 MB (1×)	312 ms +0%
Rank 1	93.6% —	4 MB (243×)	229 ms −26%
Rank 2	94.4% —	8 MB (136×)	239 ms −23%
Rank 4	94.5% —	14 MB (72×)	260 ms −16%

Language modeling — LSTM on WIKITEXT-2

Algorithm	Test perplexity	Data sent per epoch	Time per batch
SGD	91 —	7730 MB (1×)	300 ms +0%
Rank 1	102 —	25 MB (310×)	131 ms −56%
Rank 2	93 —	38 MB (203×)	141 ms −53%
Rank 4	91 —	64 MB (120×)	134 ms −55%

Asynchronous Optimization - DiLoCo

Algorithm 1 DiLoCo Algorithm

Require: Initial model $\theta^{(0)}$

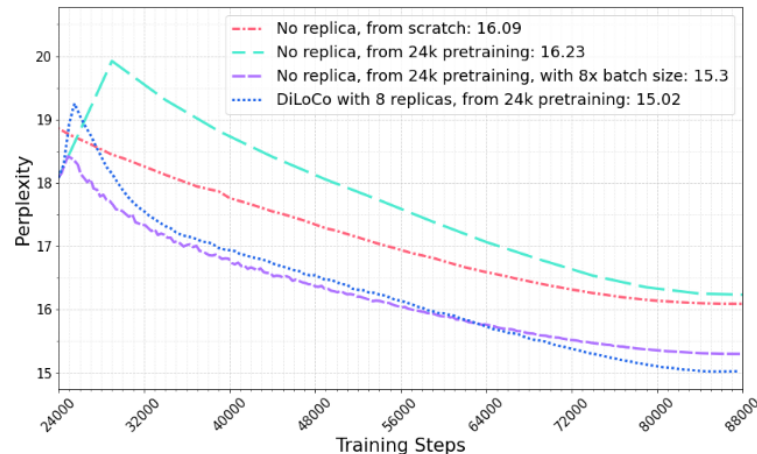
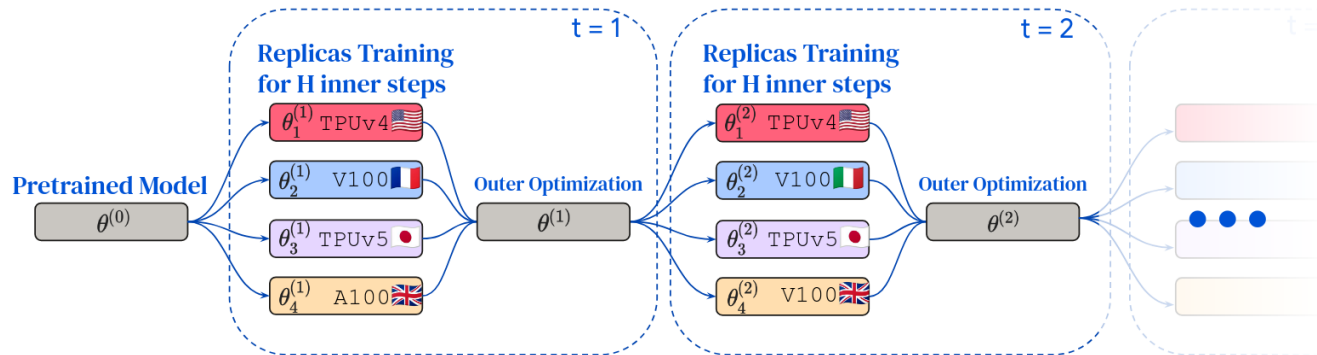
Require: k workers

Require: Data shards $\{\mathcal{D}_1, \dots, \mathcal{D}_k\}$

Require: Optimizers InnerOpt and OuterOpt

```

1: for outer step  $t = 1 \dots T$  do
2:   for worker  $i = 1 \dots k$  do
3:      $\theta_i^{(t)} \leftarrow \theta^{(t-1)}$ 
4:     for inner step  $h = 1 \dots H$  do
5:        $x \sim \mathcal{D}_i$ 
6:        $\mathcal{L} \leftarrow f(x, \theta_i^{(t)})$ 
7:        $\triangleright$  Inner optimization:
8:        $\theta_i^{(t)} \leftarrow \text{InnerOpt}(\theta_i^{(t)}, \nabla_{\mathcal{L}})$ 
9:     end for
10:   end for
11:    $\triangleright$  Averaging outer gradients:
12:    $\Delta^{(t)} \leftarrow \frac{1}{k} \sum_{i=1}^k (\theta^{(t-1)} - \theta_i^{(t)})$ 
13:    $\triangleright$  Outer optimization:
14:    $\theta^{(t)} \leftarrow \text{OuterOpt}(\theta^{(t-1)}, \Delta^{(t)})$ 
15: end for
  
```



New optimizers

New trend : optimizers learning ◀

New optimizers Abyss ◀

LION : example of a new approach ◀

New trend : optimizers learning



Learning to Optimize Neural Nets

Ke Li¹ Jitendra Malik¹

VeLO: Training Versatile Learned Optimizers

Luke Metz[‡], James Harrison[‡], C. Daniel Freeman, Amil Merchant,
Lucas Beyer, James Bradbury, Naman Agarwal, Ben Poole,
Igor Mordatch, Adam Roberts, Jascha Sohl-Dickstein[‡]

Google Research, Brain Team

Neural Optimizer Search with Reinforcement Learning

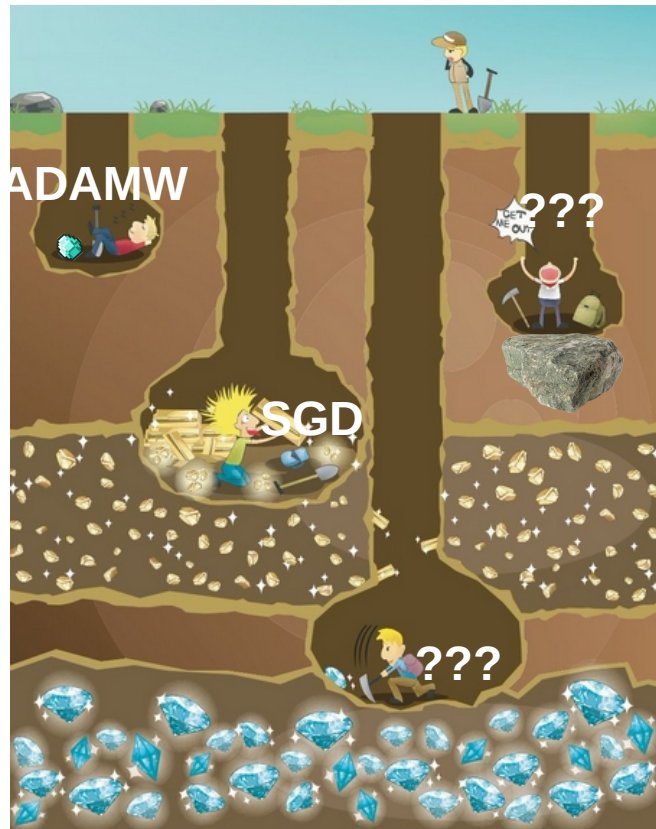
Irwan Bello^{*1}, Barret Zoph^{*1}, Vijay Vasudevan¹, Quoc V. Le¹

Learning to learn by gradient descent by gradient descent

Marcin Andrychowicz², Misha Denil¹, Sergio Gómez Colmenarejo¹, Matthew W. Hoffman¹,
David Pfau¹, Tom Schaul¹, Brendan Shillingford^{1,2}, Nando de Freitas^{1,2,3}

¹Google DeepMind ²University of Oxford ³Canadian Institute for Advanced Research
marcin.andrychowicz@gmail.com
{ndenil, sergomez, mshoffman, pfau, schaul}@google.com
brendan.shillingford@cs.ox.ac.uk, nandodefreitas@google.com

Accession	Name	Ref.	Name	Ref.
AcadGEM	Li et al. (2016)	14	Agarose	Wang et al. (2016)
ADCC	Li et al. (2016)	14	AgRIP (AgRIP2, AgRIP3)	Wang et al. (2016)
Adi	Li et al. (2016)	14	AgRIP2	Wang et al. (2016)
AdiAdh	(Owens et al. 2007)	15	AgRIP3	Kim et al. (2016)
AdiAdh2	(Owens et al. 2007)	15	AgRIP4	Kim et al. (2016)
AdiAdh3	(Owens et al. 2007)	15	AgRIP5	Kim et al. (2016)
AdiAdh4	(Owens et al. 2007)	15	AgRIP6	Kim et al. (2016)
AdiAdh5	(Owens et al. 2007)	15	AgRIP7	Kim et al. (2016)
AdiAdh6	(Owens et al. 2007)	15	AgRIP8	Kim et al. (2016)
AdiAdh7	(Owens et al. 2007)	15	AgRIP9	Kim et al. (2016)
AdiAdh8	(Owens et al. 2007)	15	AgRIP10	Kim et al. (2016)
AdiAdh9	(Owens et al. 2007)	15	AgRIP11	Kim et al. (2016)
AdiAdh10	(Owens et al. 2007)	15	AgRIP12	Kim et al. (2016)
AdiAdh11	(Owens et al. 2007)	15	AgRIP13	Kim et al. (2016)
AdiAdh12	(Owens et al. 2007)	15	AgRIP14	Kim et al. (2016)
AdiAdh13	(Owens et al. 2007)	15	AgRIP15	Kim et al. (2016)
AdiAdh14	(Owens et al. 2007)	15	AgRIP16	Kim et al. (2016)
AdiAdh15	(Owens et al. 2007)	15	AgRIP17	Kim et al. (2016)
AdiAdh16	(Owens et al. 2007)	15	AgRIP18	Kim et al. (2016)
AdiAdh17	(Owens et al. 2007)	15	AgRIP19	Kim et al. (2016)
AdiAdh18	(Owens et al. 2007)	15	AgRIP20	Kim et al. (2016)
AdiAdh19	(Owens et al. 2007)	15	AgRIP21	Kim et al. (2016)
AdiAdh20	(Owens et al. 2007)	15	AgRIP22	Kim et al. (2016)
AdiAdh21	(Owens et al. 2007)	15	AgRIP23	Kim et al. (2016)
AdiAdh22	(Owens et al. 2007)	15	AgRIP24	Kim et al. (2016)
AdiAdh23	(Owens et al. 2007)	15	AgRIP25	Kim et al. (2016)
AdiAdh24	(Owens et al. 2007)	15	AgRIP26	Kim et al. (2016)
AdiAdh25	(Owens et al. 2007)	15	AgRIP27	Kim et al. (2016)
AdiAdh26	(Owens et al. 2007)	15	AgRIP28	Kim et al. (2016)
AdiAdh27	(Owens et al. 2007)	15	AgRIP29	Kim et al. (2016)
AdiAdh28	(Owens et al. 2007)	15	AgRIP30	Kim et al. (2016)
AdiAdh29	(Owens et al. 2007)	15	AgRIP31	Kim et al. (2016)
AdiAdh30	(Owens et al. 2007)	15	AgRIP32	Kim et al. (2016)
AdiAdh31	(Owens et al. 2007)	15	AgRIP33	Kim et al. (2016)
AdiAdh32	(Owens et al. 2007)	15	AgRIP34	Kim et al. (2016)
AdiAdh33	(Owens et al. 2007)	15	AgRIP35	Kim et al. (2016)
AdiAdh34	(Owens et al. 2007)	15	AgRIP36	Kim et al. (2016)
AdiAdh35	(Owens et al. 2007)	15	AgRIP37	Kim et al. (2016)
AdiAdh36	(Owens et al. 2007)	15	AgRIP38	Kim et al. (2016)
AdiAdh37	(Owens et al. 2007)	15	AgRIP39	Kim et al. (2016)
AdiAdh38	(Owens et al. 2007)	15	AgRIP40	Kim et al. (2016)
AdiAdh39	(Owens et al. 2007)	15	AgRIP41	Kim et al. (2016)
AdiAdh40	(Owens et al. 2007)	15	AgRIP42	Kim et al. (2016)
AdiAdh41	(Owens et al. 2007)	15	AgRIP43	Kim et al. (2016)
AdiAdh42	(Owens et al. 2007)	15	AgRIP44	Kim et al. (2016)
AdiAdh43	(Owens et al. 2007)	15	AgRIP45	Kim et al. (2016)
AdiAdh44	(Owens et al. 2007)	15	AgRIP46	Kim et al. (2016)
AdiAdh45	(Owens et al. 2007)	15	AgRIP47	Kim et al. (2016)
AdiAdh46	(Owens et al. 2007)	15	AgRIP48	Kim et al. (2016)
AdiAdh47	(Owens et al. 2007)	15	AgRIP49	Kim et al. (2016)
AdiAdh48	(Owens et al. 2007)	15	AgRIP50	Kim et al. (2016)
AdiAdh49	(Owens et al. 2007)	15	AgRIP51	Kim et al. (2016)
AdiAdh50	(Owens et al. 2007)	15	AgRIP52	Kim et al. (2016)
AdiAdh51	(Owens et al. 2007)	15	AgRIP53	Kim et al. (2016)
AdiAdh52	(Owens et al. 2007)	15	AgRIP54	Kim et al. (2016)
AdiAdh53	(Owens et al. 2007)	15	AgRIP55	Kim et al. (2016)
AdiAdh54	(Owens et al. 2007)	15	AgRIP56	Kim et al. (2016)
AdiAdh55	(Owens et al. 2007)	15	AgRIP57	Kim et al. (2016)
AdiAdh56	(Owens et al. 2007)	15	AgRIP58	Kim et al. (2016)
AdiAdh57	(Owens et al. 2007)	15	AgRIP59	Kim et al. (2016)
AdiAdh58	(Owens et al. 2007)	15	AgRIP60	Kim et al. (2016)
AdiAdh59	(Owens et al. 2007)	15	AgRIP61	Kim et al. (2016)
AdiAdh60	(Owens et al. 2007)	15	AgRIP62	Kim et al. (2016)
AdiAdh61	(Owens et al. 2007)	15		



LION : example of a new approach

Algorithm 1 AdamW Optimizer

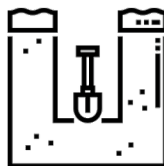
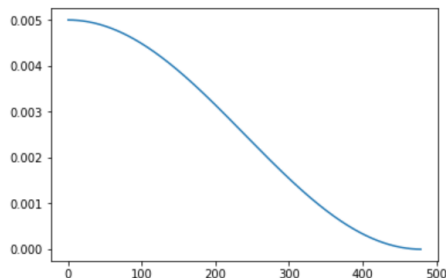
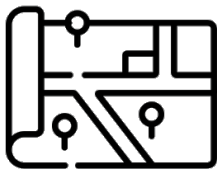
```

given  $\beta_1, \beta_2, \epsilon, \lambda, \eta, f$ 
initialize  $\theta_0, m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
while  $\theta_t$  not converged do
   $t \leftarrow t + 1$ 
   $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
  update EMA of  $g_t$  and  $g_t^2$ 
   $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
   $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
  bias correction
   $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
  update model parameters
   $\theta_t \leftarrow \theta_{t-1} - \eta_t (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1})$ 
end while
return  $\theta_t$ 
  
```

Algorithm 2 Lion Optimizer (ours)

```

given  $\beta_1, \beta_2, \lambda, \eta, f$ 
initialize  $\theta_0, m_0 \leftarrow 0$ 
while  $\theta_t$  not converged do
   $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
  update model parameters
   $c_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
   $\theta_t \leftarrow \theta_{t-1} - \eta_t (\text{sign}(c_t) + \lambda \theta_{t-1})$ 
  update EMA of  $g_t$ 
   $m_t \leftarrow \beta_2 m_{t-1} + (1 - \beta_2) g_t$ 
end while
return  $\theta_t$ 
  
```

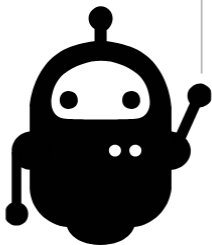


Pratice : Learning rate + Optimiseurs



Goals :

- Edit the **learning rate scheduler**
- Edit the **optimizer**
- Do training with **large batches**



From **JupyterHub**:

- Launch an interactive instance
- Go to the `tp_optimizers` folder
- Open the `DLO-JZ_Optimizers` notebook